

Rapport de conception — Entrepôt de Données de Santé

CHU · Module Big Data M2 · Épreuve E05 · BC05, compétences C27 → C31

Ce rapport couvre l'intégralité du rendu. La **partie 1** expose le besoin, l'architecture et ce qu'elle produit — indicateurs, tableaux de bord, limites. La **partie 2** traite l'automatisation, la journalisation, la traçabilité et la conduite au quotidien. La **partie 3** traite l'évolution demandée par le CHU après la première livraison. Le rapport se clôt sur la validation des chiffres, puis sur ce que le projet nous a appris.

Le dépôt lui-même est décrit dans le [README](#).

- Partie 1 — L'interface d'analyse
- 1. Le besoin
- 2. Les sources
 - 2.1 Ce que le CHU dépose
 - 2.2 Une exploration avant toute décision
- 3. L'architecture
 - 3.1 Schéma
 - 3.2 Le traitement s'exécute dans le moteur
 - 3.3 Incrémental en amont, recalcul en aval
 - 3.4 Un cloisonnement physique, pas conventionnel
 - 3.5 Traçabilité
- 4. Les traitements
 - 4.1 La pseudonymisation, au plus tôt possible
 - 4.2 Le risque de ré-identification, mesuré puis corrigé
 - 4.3 Rien n'est perdu silencieusement
 - 4.4 Ce qui change à chaque passage de couche
- 5. Le modèle
- 6. Les indicateurs
- 7. Les visualisations
 - 7.1 À quelle question répond chaque carte
 - 7.2 Pourquoi ces formes, et pas d'autres
 - 7.3 Les réserves sont dans les tableaux de bord, pas seulement ici
 - 7.4 Pourquoi la restitution n'affaiblit pas le cloisonnement
- 8. Limites et recommandations
 - 8.1 Limites
 - 8.2 Recommandations
- Partie 2 — L'automatisation
- 9. Ce qui déclenche le pipeline
- 10. Ce qui se passe quand ça échoue
- 11. La journalisation
- 12. La traçabilité, de bout en bout
- 13. Utilisation et maintenance
- Partie 3 — L'évolution du besoin
- 14. La demande
 - 14.1 Ce que le CHU demande, et ce qui change
 - 14.2 Un référentiel n'est pas un jour de dépôt
 - 14.3 « Sans retraiter l'existant » — la propriété était déjà là
- 15. Les deux pièges

- 15.1 Le premier piège : un référentiel incomplet
- 15.2 Le second piège : le service vient du séjour
- 16. Le modèle après l'évolution
 - 16.1 Un quatrième fait, une dimension de plus
 - 16.2 La qualité des actes, démontrée sur des règles qui ne servent pas
- 17. Les nouveaux indicateurs
 - 17.1 Ce qu'ils portent, et ce qu'ils prouvent
 - 17.2 Ce qu'il a fallu accepter
- Partie 4 — Le déploiement cloud
- 18. Ce qui est déployé, et pourquoi là
 - 18.1 Le même entrepôt, ailleurs
 - 18.2 AKS plutôt que Container Apps
 - 18.3 Blob plutôt qu'un volume partagé
- 19. La sécurité, dans l'ordre où elle est prononcée
- 20. La nuit, sans superviseur
- 21. Limites et coût
- Validation des chiffres
- L'équation de conservation, couche par couche
- Les indicateurs, confrontés à leur recalcul depuis les faits
- Confrontation aux valeurs de référence de l'intervenant
- Ce que cette validation ne couvre pas
- Ce que ce projet nous a appris
- Un contrôle peut être juste et ne rien contrôler
- Une règle qui n'attrape rien doit quand même être démontrée
- Certains défauts ne se voient qu'en ouvrant le produit fini
- Une montée de version n'est pas réversible
- Un client ne doit pas supposer le format de ce qu'il reçoit
- Une remise en état ne remet en état que ce qu'elle sait recharger
- Un avertissement qui décrit le normal cesse d'être lu
- Ne rien casser ne coûte rien — si l'on s'y est préparé avant

Sommaire

Rapport de conception — Entrepôt de Données de Santé

Partie 1 — L'interface d'analyse

1. Le besoin
2. Les sources
 - 2.1 Ce que le CHU dépose
 - 2.2 Une exploration avant toute décision
3. L'architecture
 - 3.1 Schéma
 - 3.2 Le traitement s'exécute dans le moteur
 - 3.3 Incrémental en amont, recalcul en aval
 - 3.4 Un cloisonnement physique, pas conventionnel
 - 3.5 Traçabilité
4. Les traitements
 - 4.1 La pseudonymisation, au plus tôt possible
 - 4.2 Le risque de ré-identification, mesuré puis corrigé
 - 4.3 Rien n'est perdu silencieusement
 - 4.4 Ce qui change à chaque passage de couche
 - 4.5 Ce que le lake a le droit de contenir
5. Le modèle
6. Les indicateurs
7. Les visualisations
 - 7.1 À quelle question répond chaque carte
 - 7.2 Pourquoi ces formes, et pas d'autres
 - 7.3 Les réserves sont dans les tableaux de bord, pas seulement ici
 - 7.4 Pourquoi la restitution n'affaiblit pas le cloisonnement
8. Limites et recommandations
 - 8.1 Limites
 - 8.2 Recommandations

Partie 2 — L'automatisation

9. Ce qui déclenche le pipeline
10. Ce qui se passe quand ça échoue

- 11. La journalisation
 - 12. La traçabilité, de bout en bout
 - 13. Utilisation et maintenance
-

Partie 3 — L'évolution du besoin

- 14. La demande
 - 14.1 Ce que le CHU demande, et ce qui change
 - 14.2 Un référentiel n'est pas un jour de dépôt
 - 14.3 « Sans retraiter l'existant » — la propriété était déjà là
 - 15. Les deux pièges
 - 15.1 Le premier piège : un référentiel incomplet
 - 15.2 Le second piège : le service vient du séjour
 - 16. Le modèle après l'évolution
 - 16.1 Un quatrième fait, une dimension de plus
 - 16.2 La qualité des actes, démontrée sur des règles qui ne servent pas
 - 17. Les nouveaux indicateurs
 - 17.1 Ce qu'ils portent, et ce qu'ils prouvent
 - 17.2 Ce qu'il a fallu accepter
 - 17.3 La règle posée plutôt que le cas traité
-

Partie 4 — Le déploiement cloud

- 18. Ce qui est déployé, et pourquoi là
 - 18.1 Le même entrepôt, ailleurs
 - 18.2 AKS plutôt que Container Apps
 - 18.3 Blob plutôt qu'un volume partagé
 - 19. La sécurité, dans l'ordre où elle est prononcée
 - 20. La nuit, sans superviseur
 - 21. Limites et coût
-

Validation des chiffres

- L'équation de conservation, couche par couche
 - Les indicateurs, confrontés à leur recalcul depuis les faits
 - Confrontation aux valeurs de référence de l'intervenant
 - Ce que cette validation ne couvre pas
-

Ce que ce projet nous a appris

- Un contrôle peut être juste et ne rien contrôler

Une règle qui n'attrape rien doit quand même être démontrée

Certains défauts ne se voient qu'en ouvrant le produit fini

Une montée de version n'est pas réversible

Un client ne doit pas supposer le format de ce qu'il reçoit

Une remise en état ne remet en état que ce qu'elle sait recharger

Un avertissement qui décrit le normal cesse d'être lu

Ne rien casser ne coûte rien — si l'on s'y est préparé avant

Partie 1 — L'interface d'analyse

Le besoin, les sources, l'architecture et ce qu'elle produit : les indicateurs, les tableaux de bord, et les limites de ce qui est livré.

1. Le besoin

Le CHU dispose de données éparpillées entre quatre systèmes — dossier patient, urgences, laboratoire, monitoring des chambres — exportées chaque jour dans des formats hétérogènes (CSV, JSON imbriqué, Parquet). Personne ne peut aujourd'hui répondre à une question transverse sans consolider ces exports à la main.

Deux publics, aux besoins opposés :

	Attend	Granularité requise
Direction hospitalière	Piloter l'activité et la qualité des soins	Fine, par service et par jour
Recherche clinique	Décrire des cohortes de patients	Agrégée, jamais individuelle

Six indicateurs sont nommés : durée moyenne de séjour par service, passages aux urgences par jour, taux de ré-admission à 30 jours, relevés de constantes en alerte, prévalence par pathologie, distribution par âge et sexe. Le § 4 du sujet ajoute une septième ligne, ouverte — « toute autre vue d'activité pertinente » — que quatre vues complémentaires remplissent : occupation, mortalité, case-mix, origine géographique.

Les six se calculent depuis la couche gold, et `tests.verifier_indicateurs` les restitue à chaque exécution — la valeur obtenue **et** la propriété qui la fonde. Un chiffre qui s'affiche ne prouve rien par lui-même : ce qui le rend opposable, c'est que son dénominateur, son périmètre et ses seuils soient vérifiés en même temps que lui. Les valeurs constatées sur le dépôt courant figurent au § 6.

Chaque public reçoit désormais ses indicateurs par un tableau de bord Metabase qui lui est propre — « Pilotage hospitalier » pour la direction, « Recherche clinique » pour la recherche — provisionné par `eds.restitution` (§ 1 du sujet). Aucun réglage de l'outil de restitution n'intervient dans le cloisonnement : chaque tableau de bord interroge la couche gold à travers le compte ClickHouse borné de son usage, et c'est ce compte, pas Metabase, qui prononce le refus si une carte tentait d'atteindre l'autre base. Le choix de l'outil et la façon dont il s'articule au cloisonnement sont détaillés au § 7.4.

2. Les sources

2.1 Ce que le CHU dépose

Le CHU dépose chaque jour ses exports dans un espace en **lecture seule** (`source-filestorage/`), dans trois formats différents. Vingt-huit jours de dépôt sont disponibles, du 1er au 28 août 2026.

Source	Format	Dépôts	Volume brut	Ce qu'elle porte
patients	CSV	3 jours	18 000 lignes → 6 000 patients	patient_id, nir, nom, prenom, birth_date, sex, region_code — l'identité réelle
sejours	CSV	28 jours	6 797 séjours	stay_id, patient_id, service_code, admission et sortie, modes d'entrée et de sortie
diagnostics	JSON imbriqué	28 jours	6 797 objets → 12 720 codes	un à trois codes CIM-10 par séjour, typés principal ou associe
monitoring	Parquet	28 jours	41 778 relevés	constantes au chevet — fréquence cardiaque, saturation, température. La source volumineuse
referentiels	CSV	1er jour	8 services · 13 codes CIM-10	nomenclatures : code → libellé

Quatre traits de ces sources ont déterminé l'architecture, et aucun n'est annoncé par le cahier des charges — ils ont été découverts en les profilant (§ 2.2). Le détail chiffré de ce profilage, anomalie par anomalie, est dans [exploration/RAPPORT-EXPLORATION.md](#).

La complétude n'est pas uniforme. Le monitoring ne couvre que **12,8 % des séjours**, et seulement deux services sur huit en sont équipés. Ce n'est pas une anomalie à corriger mais un périmètre à énoncer partout où l'indicateur de constantes est diffusé.

La contrainte qui structure tout le projet : il s'agit de données de santé, catégorie particulière au sens de l'article 9 du RGPD. Les fichiers sources contiennent l'identité réelle des patients — nom, prénom, numéro de sécurité sociale, date de naissance complète. La conformité n'est donc pas une couche ajoutée en fin de chaîne : c'est une contrainte de conception présente à chaque étape, et elle explique la majorité des choix ci-dessous.

2.2 Une exploration avant toute décision

Cinq sources ont été profilées avant d'écrire la moindre ligne de pipeline. Ce travail a produit quatre constats qui ont directement déterminé l'architecture, et qu'aucune lecture du cahier des charges ne laissait deviner.

patients est un snapshot cumulatif, les trois autres sources sont des deltas. Chaque fichier journalier de patients contient toute la population connue à date : 18 000 lignes pour 6 000 patients réels. Les séjours, diagnostics et relevés n'apparaissent au contraire qu'une fois. Appliquer la même stratégie d'ingestion aux quatre sources aurait multiplié par 3 tout indicateur rapporté au patient.

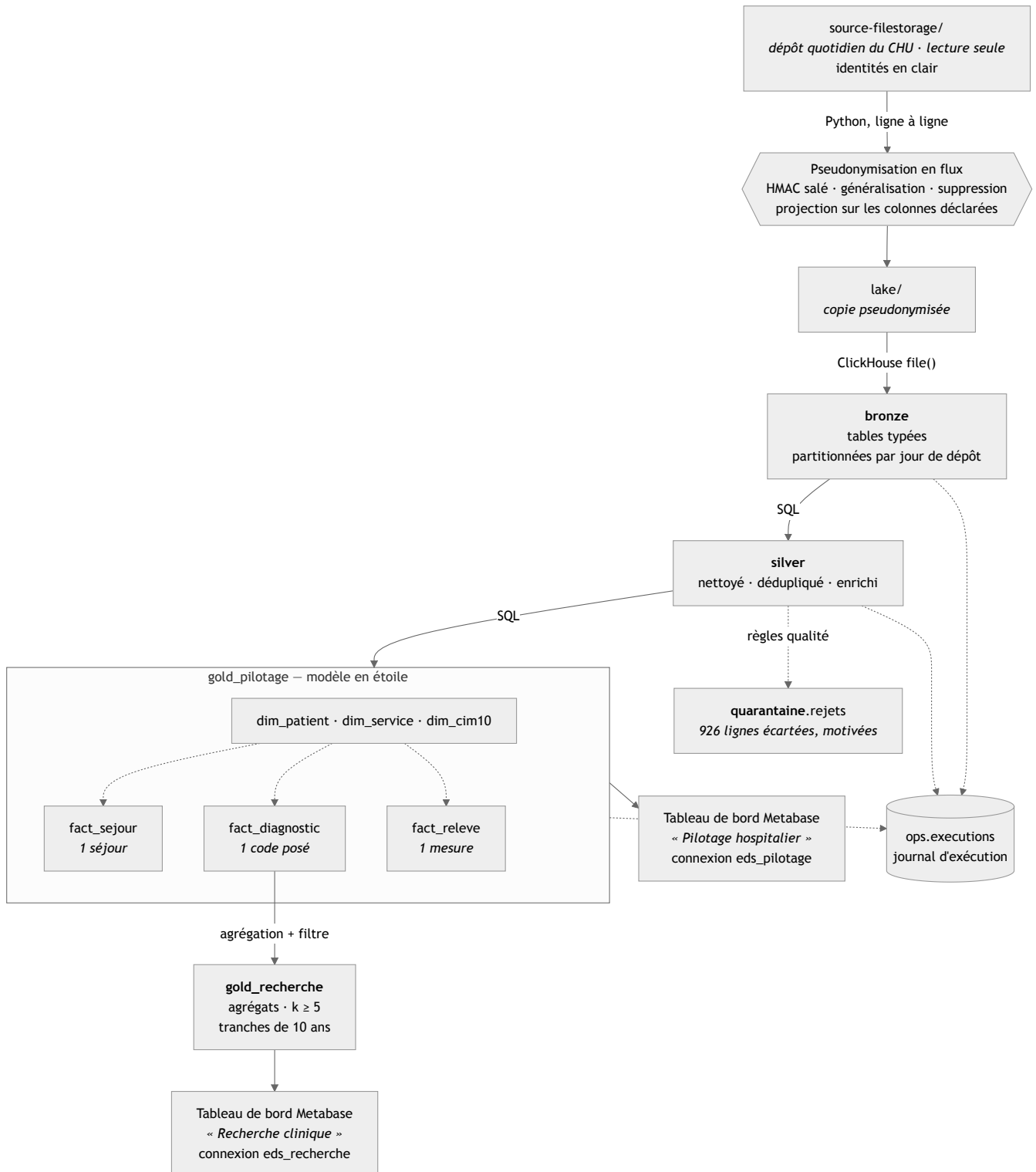
Les sources n'ont pas le même calendrier de dépôt. Séjours, diagnostics et relevés sont déposés les 28 jours du mois ; le snapshot **patients** ne l'est que les trois derniers. La population de référence est donc l'**union** des dépôts, et non celui du jour : un séjour du 1er août n'est décrit que par un fichier arrivé 25 jours plus tard. Une lecture jour à jour prendrait ce décalage pour une rupture d'intégrité référentielle.

Le monitoring déborde de son jour de dépôt. Le fichier du 1er août contient des relevés jusqu'au 3, et le décalage se reproduit sur les 28 dépôts. Agréger sur le jour de dépôt attribuerait au 1er des alertes survenues deux jours plus tard : l'agrégation se fait donc sur l'horodatage de la mesure.

Les valeurs aberrantes du monitoring sont une panne de capteur, pas du bruit. Fréquence cardiaque et saturation sont *toujours* invalides ensemble — 858 fois les deux, jamais l'une seule — sur exactement quatre combinaisons de butée : (0 ou 500 bpm) × (0 ou 120 %). Le relevé entier est écarté : un capteur déconnecté ne garantit la fiabilité d'aucune de ses mesures.

3. L'architecture

3.1 Schéma



Les identités n'existent que dans la source, fournie et non versionnée. La frontière de pseudonymisation est franchie **une seule fois**, en Python, avant toute écriture persistante.

3.2 Le traitement s'exécute dans le moteur

Le lake est monté en lecture seule dans ClickHouse, qui lit lui-même les fichiers CSV, JSON et Parquet via `file()`. **Python n'envoie que du SQL** : aucune donnée ne transite par sa mémoire. C'est l'inverse de l'anti-pattern qui consiste à extraire les données pour les transformer avec pandas.

3.3 Incrémental en amont, recalcul en aval

Bronze est partitionné par jour de dépôt : rejouer un jour supprime sa partition et la réécrit, sans toucher aux autres. Silver et gold sont au contraire **recalculés intégralement** à chaque exécution — 0,12 seconde à ce volume.

Ce choix garantit qu'un rejeu produit exactement le même état, avec un mécanisme explicable en une phrase. Sa limite est réelle et assumée : il ne tiendrait pas sur plusieurs années d'historique (voir § 8.1).

3.4 Un cloisonnement physique, pas conventionnel

Le sujet définit **deux publics** et ce que chacun doit voir : au pilotage la durée de séjour, l'activité des urgences, la réadmission et la surveillance des constantes ; à la recherche la prévalence par pathologie et la description de cohorte. Il en tire une contrainte explicite — « pilotage et recherche ne voient pas les mêmes données → droits d'accès distincts ».

Un filtre applicatif serait contournable par quiconque écrit sa propre requête. Nous avons donc séparé **physiquement** : deux bases ClickHouse, deux comptes de lecture, des droits disjoints.

Quatre comptes ClickHouse, aux vocations distinctes :

Compte	Vocation	Ce qu'il peut atteindre
<code>eds_admin</code>	Exécuter le pipeline — créer les tables, appliquer les habilitations	Tout. C'est le compte du traitement, pas un compte d'usage
<code>eds_pilotage</code>	Direction hospitalière — piloter l'activité et la qualité des soins	17 colonnes de <code>gold_pilotage</code> . Rien d'autre
<code>eds_recherche</code>	Recherche clinique — décrire des cohortes, sous contrainte de petits effectifs	Les deux tables d'agrégats de <code>gold_recherche</code> . Rien d'autre
<code>eds_exploitation</code>	Investigation technique — incident, piste d'audit, demande d'effacement	<code>bronze</code> , <code>silver</code> , <code>quarantaine</code> , <code>ops</code> — en lecture seule

Le compte d'investigation n'est pas une commodité technique. Dans un entrepôt de données de santé, quelqu'un doit pouvoir **remonter à la ligne d'origine** — pour diagnostiquer un incident, produire une piste d'audit, ou exécuter une demande d'effacement au titre du RGPD. C'est précisément ce que les colonnes `_fichier_source` et `_run_id` rendent possible. Ni le pilotage ni la recherche n'atteignent ce détail, fût-il pseudonymisé : eux n'ont accès qu'à gold, et seulement à leur base.

Ce compte mérite d'être justifié. L'administration aurait pu investiguer avec le compte du pipeline, qui a tous les droits. Nous ne l'avons pas fait : `eds_admin` peut créer et supprimer des bases, et ce pouvoir n'a pas sa place dans un usage quotidien, où une requête maladroite suffirait. Le compte d'exploitation applique le moindre privilège — il lit tout ce qui est nécessaire à une investigation, et le moteur lui refuse toute écriture.

Les droits sont posés colonne par colonne, pas base par base. C'est la conséquence directe du choix d'un modèle en étoile : la couche gold contenant les faits au grain de l'événement, un `GRANT` sur la base entière donnerait au pilotage l'accès à `patient_pseudo` et à `stay_id`. Or la direction consulte des indicateurs d'activité — elle n'a jamais à désigner un patient ni à relier deux séjours.

`eds_pilotage` ne dispose donc que des **17 colonnes** que ses indicateurs utilisent : codes de service, dates, tranches d'âge, durées et drapeaux d'alerte. Ni le pseudonyme, ni l'identifiant de séjour, ni les horodatages précis, ni les constantes brutes ; `dim_patient` et `fact_diagnostic` ne lui sont pas accordées du tout. Vérifié : il ne peut ni lire le pseudonyme, ni dénombrer des patients, ni exécuter un `SELECT *` — et il calcule sans difficulté la DMS, les passages aux urgences et les relevés en alerte.

http://localhost:8123 v25.8.33.6, uptime 3 hr eds_pilotage

```
SELECT service_code, duree_jours FROM gold_pilotage.fact_sejour LIMIT 5;
```

Run (Ctrl/Cmd+Enter) 5 rows in result, 0.00 sec. 100.0%, Read 6.73 thousand rows, 67.40 KB (3.20 million/sec, 32.07 MB/sec)

#	service_code	duree_jours
1	CARDIO	6.291666666666667
2	CARDIO	4
3	CARDIO	2.7916666666666665
4	CARDIO	6.625
5	CARDIO	2.125

ClickHouse

http://localhost:8123 v25.8.33.6, uptime 3 hr eds_pilotage

```
SELECT patient_pseudo FROM gold_pilotage.fact_sejour LIMIT 5;
```

Run (Ctrl/Cmd+Enter)

Code: 497. DB::Exception: eds_pilotage: Not enough privileges. To execute this query, it's necessary to have the grant SELECT(patient_pseudo) ON gold_pilotage.fact_sejour. (ACCESS_DENIED) (version 25.8.33.6 (official build))

Le même compte, la même table, et une colonne refusée. Le moteur nomme précisément ce qui manque — `grant SELECT(patient_pseudo) ON gold_pilotage.fact_sejour` — ce qui rend la borne vérifiable plutôt que déclarative. Un `GRANT` posé base par base n'aurait pas produit ce refus.

Cette borne ne dépend pas de qui interroge, mais du compte employé. Quiconque se connecte avec `eds_pilotage` — administrateur compris — se voit opposer le même refus, prononcé par le moteur.



La même session, deux requêtes : l'une passe, l'autre est refusée en `ACCESS_DENIED`. Aucune application n'intervient entre les deux — c'est la console SQL de ClickHouse.

C'est ce qui distingue un cloisonnement d'un réglage d'interface : l'outil de restitution branché sur ces bases — Metabase, § 7.4 — hérite de la borne sans pouvoir la contourner, quels que soient ses propres réglages. Ce n'est plus une intention : chaque public **reçoit désormais ses indicateurs par un tableau de bord Metabase dédié**, et la borne que l'interface applique est celle du compte ClickHouse qu'elle emploie, jamais un paramètre posé dans Metabase lui-même.

Ce choix conduit à faire des indicateurs des **tables matérialisées et non des vues**. Une vue ordinaire s'exécute en ClickHouse avec les droits de l'appelant : une vue gold obligerait à ouvrir aussi l'accès à silver, et le cloisonnement s'effondrerait. Le moteur sait, depuis la version 24.4, déclarer une vue `SQL SECURITY DEFINER` qui s'exécute avec les droits de son créateur ; cette alternative a été écartée pour une autre raison — un indicateur recalculé à chaque lecture n'est pas un instantané, et c'est l'instantané qui rend possible le contrôle de `tests.verifier_indicateurs`, qui confronte chaque table au fait dont elle sort.

3.5 Traçabilité

Chaque ligne de bronze et de silver porte **le jour de dépôt, le fichier dont elle provient** et l'identifiant du run qui l'a produite, plus son horodatage — d'ingestion en bronze, de construction en silver. La question « *cette donnée, d'où vient-elle et quand a-t-elle été traitée ?* » se répond donc en une requête, sans jointure entre couches.

La propriété vaut aussi pour ce qui a été **écarté** : `quarantaine.rejets` porte la même provenance. On peut donc remonter d'un problème de qualité au fichier de dépôt qui l'a introduit — c'est ce que demande une investigation réelle.

Trois nuances assumées. En silver, `patients` est une réduction de plusieurs lignes bronze (snapshot cumulatif) : sa provenance est celle de la **version retenue**, d'où les colonnes `_jour_depot_retenu` et `_fichier_source_retenu`. Pour `monitoring`, `_jour_depot` désigne le jour du fichier, jamais celui de la mesure — les deux diffèrent, le flux débordant de son jour de dépôt.

Enfin, les deux **référentiels** (`ref_services`, `ref_cim10`) portent le fichier d'origine mais pas de colonne `_jour_depot`, et ne sont pas partitionnés. Ce ne sont pas des données journalières : ils sont rechargés en entier à chaque exécution, et leur chemin de fichier — `lake/referentiels/2026-08-01/services.csv` — contient déjà le jour. Une colonne de plus n'aurait rien ajouté qu'un doublon.

Gold s'arrête volontairement à `_run_id` et `_built_at`. La provenance fichier y serait inexploitable : le compte de pilotage n'a pas même le droit de lire `stay_id`, et une investigation passe par la connexion d'exploitation, donc par silver et bronze. Ce qu'on veut savoir de gold, c'est quelle exécution l'a produit — pas de quel dépôt vient une moyenne.

La table `ops.executions` enregistre chaque étape — durée, volume, statut, cause d'échec. Aucune donnée de santé n'entre dans ce journal.

4. Les traitements

4.1 La pseudonymisation, au plus tôt possible

Le cahier des charges demande de pseudonymiser « à l'entrée du lake » tout en décrivant par ailleurs le lake comme une « copie brute ». Ces deux exigences sont incompatibles. **Nous avons tranché en faveur de la minimisation RGPD** : le lake contient une copie fidèle mais pseudonymisée.

La transformation s'applique **au fil de la copie, ligne par ligne**. Les identités ne sont donc écrites nulle part — pas même dans un répertoire temporaire — et l'empreinte mémoire ne dépend pas de la taille des fichiers.

Donnée source	Traitement	Justification
<code>patient_id</code> (IPP)	HMAC-SHA256 salé, tronqué à 64 bits	Déterministe pour préserver les jointures, salé parce que l'espace des IPP est énumérable — un SHA-256 nu serait cassable par dictionnaire
<code>nir</code> , <code>nom</code> , <code>prenom</code>	Supprimés	Directement identifiants, sans usage pour les indicateurs demandés
<code>birth_date</code>	Généralisée à l'année	Aucun indicateur ne requiert la date exacte

Un contrôle automatisé rejoue les **17 384 valeurs identifiantes** de la source contre l'intégralité du lake et échoue si l'une d'elles y apparaît. Il vérifie également l'absence de collision de pseudonyme et la préservation des jointures.



Le contrôle le plus direct, exécuté avec le compte d'administration qui voit tout : aucune colonne `nir`, `nom`, `prenom`, `birth_date` ni `patient_id` ne subsiste nulle part dans l'entrepôt. `empty result` — et ce n'est pas un périmètre restreint qui le produit, c'est `system.columns` interrogé sur toutes les bases.

4.2 Le risque de ré-identification, mesuré puis corrigé

La généralisation à l'année demandée par le sujet **ne suffit pas**. Mesure du k-anonymat sur les quasi-identifiants restants :

Granularité de l'âge	Population à $k \geq 5$	Patients uniques	Cohortes sous le seuil
Année de naissance	62,3 %	185	95
Tranche de 10 ans	100 %	0	13

D'où la règle retenue : l'année n'existe qu'en pilotage, accès restreint ; la base recherche n'expose que des tranches de dix ans. Le filtre des petits effectifs (≥ 5 patients) est appliqué à l'écriture, de sorte qu'aucune donnée sous le seuil n'existe dans cette base — il n'y a rien à penser à masquer au moment de la lecture.

La généralisation seule ne suffit pas, et les données le montrent. Les tranches de dix ans ramènent toute la population à $k \geq 5$, mais **treize cohortes restent sous le seuil** : la nomenclature comporte trois pathologies rares — mucoviscidose, amyotrophie spinale, trisomie 21 — dont les effectifs sont faibles quelle que soit la granularité de l'âge. C'est le filtre à l'écriture, et lui seul, qui garantit la propriété.

Ce filtre se déclenche donc sur les données réelles. Deux prévalences sont supprimées à la construction — trisomie 21 (3 patients) et mucoviscidose (4) — ainsi que treize cohortes de description : elles existent au grain du fait en pilotage, elles n'atteignent pas `gold_recherche.tests.demontrer_effectifs` va plus loin et fabrique le cas limite que les données ne fournissent pas — une pathologie portée par exactement 4 patients, une autre par exactement 5 — reconstruit silver et gold, et constate où tombe la coupe : la première est retenue, la seconde passe. Le seuil coupe bien sous 5, et non à 5, comme le demande le §5.

4.3 Rien n'est perdu silencieusement

Décision de l'intervenant : la cohérence temporelle d'un séjour n'écarte plus ses diagnostics ni ses relevés. Un séjour dont `discharge_ts < admission_ts` est toujours écarté de `silver.sejours` (règle du §3) — mais cette anomalie porte sur **deux colonnes de la table `sejours`**, et ne dit rien de la validité d'un code CIM-10 posé pendant ce séjour, ni d'une mesure prise au chevet du patient : ce sont des faits médicaux distincts, sur d'autres tables. Le calquer sur ces deux tables ferait porter à une donnée clinique valide l'anomalie d'une autre colonne, sur une autre ligne.

En conséquence, `silver.diagnostics` et `silver.monitoring` lisent désormais `bronze.sejours` directement — jamais `silver.sejours` — pour s'enrichir du patient, du service et de l'admission porteurs, et ne consultent `silver.sejours` que pour poser un drapeau `sejour_cohérent` (1 si le séjour y figure), **jamais pour filtrer**. Le diagnostic ou le relevé d'un séjour incohérent est donc **conservé** en silver, avec `sejour_cohérent = 0`, et recopié tel quel jusqu'en gold (`fact_diagnostic.sejour_cohérent`, `fact_releve.sejour_cohérent`) — signalé, jamais silencieusement perdu. Seule l'**absence de patient identifié** écarte réellement l'un ou l'autre : séjour introuvable en bronze (motif `sejour_inconnu`) ou pseudonyme vide (motif `sejour_ecarte`) — cf. le détail des motifs plus bas.

Chaque ligne écartée est écrite dans `quarantaine.rejets` avec son motif. La propriété en découle et se vérifie en une requête :

```
sejours      6 729 +    68 = 6 797
diagnostics 12 720 +     0 = 12 720
monitoring  40 920 +   858 = 41 778
```

Le total de la quarantaine monitoring (858) ne porte plus qu'un seul motif, `capteur_hors_plage`. Les motifs `sejour_ecarte`, `sejour_inconnu` et `releve_hors_sejour` valent chacun 0 ligne sur ce dépôt — ils restent définis et exercés (`tests.demontrer qualite`), mais aucune donnée réelle ne les déclenche ici : aucun `stay_id` de monitoring n'est absent de `bronze.sejours`, aucun `patient_id` n'est vide, et les 520 relevés postérieurs à une sortie de séjour incohérent échappent au contrôle temporel plutôt que de l'enfreindre (cf. § 4.4).

Pour chaque source, `silver + quarantaine = bronze`, à la ligne près. C'est ce qui distingue écarter une donnée de la perdre.

Une ligne fautive n'est pas toujours écartée : elle peut être corrigée, lorsque la valeur inutilisable ne porte aucune clé. La colonne `action` du registre sépare les deux issues — `'ecarte'` entre dans l'équation, `'corrige'` est signalée sans être soustraite. Sans cette distinction, corriger une valeur ferait disparaître une ligne du compte, ou bien la correction resterait invisible : les deux sont inacceptables dans un registre d'incidents qualité.

Pourquoi une base à part, et non `silver.rejets`. Deux raisons, et aucune n'est esthétique. D'abord le contrat de la couche : `silver` signifie « nettoyé, cohérent » ; y loger la table des lignes sales brouille exactement ce que la couche promet, et un `GRANT SELECT ON silver.*` livrait le registre avec les données propres. Ensuite le cycle de vie : ces lignes portent `stay_id` et, dans `detail`, les valeurs brutes fautives — c'est de la donnée de santé pseudonymisée, mais dont la durée de conservation n'est pas celle de l'entrepôt. On purge une quarantaine quand l'incident qualité est instruit, pas quand la donnée expire. Une base distincte permet de lui appliquer sa propre rétention et ses propres droits — et, le jour venu, de la confier à un référent qualité sans lui ouvrir `silver`.

4.4 Ce qui change à chaque passage de couche

Le sujet demande de « justifier chaque chiffre ». Voici, frontière par frontière, ce que subit la donnée — et l'effet chiffré de chaque opération.

Source → Lake · pseudonymisation

Seules `patients` et `sejours` sont transformées : ce sont les deux seules sources portant de l'identité. Mais toutes sont projetées sur les colonnes que `config.COLONNES_LAKE` les autorise à déposer — c'est le sujet du § 4.5.

Opération	Effet
<code>patient_id</code> → <code>patient_pseudo</code>	HMAC-SHA256 salé, tronqué à 64 bits. Appliqué aux deux sources avec le même sel, pour préserver la jointure
<code>birth_date</code> → <code>birth_year</code>	Généralisation. 1933-12-09 devient 1933
<code>nir</code> , <code>nom</code> , <code>prenom</code>	Supprimés — 3 colonnes sur 7 disparaissent de <code>patients</code>
Projection	Chaque fichier réduit aux colonnes déclarées, CSV, JSON et Parquet compris
Volumétrie	inchangée : 24 797 lignes entrent, 24 797 sortent

Lake → Bronze · typage et mise en forme tabulaire

Aucune ligne n'est écartée à ce stade. C'est délibéré : on ne peut compter que ce qu'on a laissé entrer.

Opération	Effet
Typage explicite	<code>String</code> → <code>DateTime</code> , <code>UInt16</code> , <code>Decimal(4,1)</code> , <code>LowCardinality</code>
<code>discharge_ts</code> vide → <code>NULL</code>	683 séjours en cours préservés comme tels, et non datés par défaut
Dates lues en mode tolérant	<code>parseDateTimeBestEffortOrNull</code> : une date illisible entre en <code>NULL</code> au lieu de faire échouer le chargement du jour entier. Un drapeau <code>_discharge_illisible</code> distingue « vide, séjour en cours » de « illisible » — que le <code>NULL</code> confondrait
Aplatissement du JSON	6 797 objets imbriqués → 12 720 lignes , une par code posé. Aucune donnée créée ni perdue
Types larges et signés	<code>Int16</code> pour la fréquence cardiaque : les 858 relevés aberrants peuvent entrer et donc être comptés
Partitionnement	Par jour de dépôt — c'est ce qui rend le rejeu d'un jour possible
Ajout de 4 colonnes techniques	<code>_jour_depot</code> , <code>_fichier_source</code> , <code>_ingested_at</code> , <code>_run_id</code>

Bronze → Silver · qualité, déduplication, enrichissement

C'est la seule frontière où des lignes sont écartées, et chacune l'est avec son motif.

Une règle décide de ce qui a le droit d'être ici. Silver applique les règles de **validité** que le sujet fournit — plages physiologiques, cohérence temporelle, déduplication. Les règles **métier**, que le sujet ne fournit pas et qui se paramètrent, appartiennent à gold : les seuils d'alerte et l'âge à l'événement n'y sont donc pas. Cette frontière n'est pas une convention de style, elle se vérifie (`tests.verifier_qualite` échoue si silver recalcule un âge ou une alerte).

Table	Entrée	Sortie	Opération
<code>patients</code>	18 000	6 000	Déduplication du snapshot cumulatif : <code>row_number()</code> sur le jour de dépôt décroissant, la ligne de rang 1 est retenue — 118 patients ont un attribut divergent entre redépôts ; sexe normalisé <code>upper(trim(...))</code> , hors M/F → <code>'inconnu'</code> ; date de naissance illisible → <code>birth_year</code> <code>NULL</code> , ligne corrigée , jamais écartée
<code>sejours</code>	6 797	6 729	–68 incohérences temporelles (<code>discharge_ts < admission_ts</code>), –0 date illisible, –0 patient manquant
<code>diagnostics</code>	12 720	12 720	aucun écart — la cohérence temporelle du séjour porteur n'écarte plus le diagnostic (décision de l'intervenant, § 4.3)
<code>monitoring</code>	41 778	40 920	–858 capteur hors plage, –0 <code>sejour_inconnu</code> , –0 <code>sejour_ecarte</code> (patient manquant), –0 <code>releve_hors_sejour</code> (évalué seulement sur séjour cohérent)
<code>qualitative</code>	—	926	Toute ligne écartée (<code>action = 'ecarte'</code>) ou corrigée (<code>'corrige'</code>), avec son motif et son détail

Pourquoi `row_number()` et non `argMax` pour déduplicuer les patients. `birth_year` est `Nullable` depuis qu'une date de naissance illisible est corrigée plutôt qu'écartée. Or `argMax(v, t)` ignore les lignes dont l'argument `v` est `NULL` — vérifié sur ClickHouse 25.8 : si la version la plus récente d'un patient porte une année illisible, `argMax` renvoie l'année d'un dépôt *antérieur*, silencieusement. La ligne retenue ne serait alors plus celle du dépôt retenu. `row_number()` classe la **ligne entière** et n'a pas ce défaut : ce qui est `NULL` au rang 1 reste `NULL`.

Les motifs de la quarantaine, un par un — y compris ceux à zéro ligne. Chaque source a ses propres motifs, mutuellement exclusifs entre eux (une ligne n'est jamais comptée deux fois) :

Motif	Source(s)	Sur ce dépôt	Pourquoi il existe malgré son compte
incoherence_temporelle	sejours	68, écartées	<code>discharge_ts < admission_ts</code> — la seule anomalie qui écarte un séjour lui-même
patient_manquant	patients , sejours	0, écartées	<code>patient_id</code> vide en source — la pseudonymisation ne produit pas de hachage bidon, donc un pseudonyme vide ne peut agréger plusieurs lignes sous une fausse personne
capteur_hors_plage	monitoring	858, écartées	FC/SpO2 hors plage physiologique — le marqueur de panne capteur identifié en exploration
sejour_ecarte	diagnostics , monitoring	0, écartées	Le séjour porteur a lui-même été écarté pour <code>patient_manquant</code> (pseudonyme vide) — seul cas qui écarte encore un diagnostic ou un relevé
sejour_inconnu	diagnostics , monitoring	0, écartées	<code>stay_id</code> absent de <code>bronze.sejours</code> — aucune trace du séjour porteur, distinct de <code>sejour_ecarte</code> (qui suppose le séjour présent mais son patient absent)
releve_hors_sejour	monitoring	0, écartées	Horodatage hors de la fenêtre [admission, sortie] d'un séjour cohérent — ne se déclenche jamais sur ce dépôt (cf. plus bas pour les 528 relevés post-sortie, qui échappent à ce contrôle)

Les quatre derniers motifs sont à zéro ligne sur ce dépôt réel, mais ils ne sont pas du code mort : chacun est exercé par `tests.demontrer qualite`, qui fabrique le cas que la source ne fournit pas (patient manquant, `stay_id` inconnu) et vérifie où tombe la ligne.

Les deux contrôles de format du §3 ne se déclenchent pas sur ces données. « Dates valides » et « sexe normalisé (M/F) » : la source est propre sur les deux — 9 045 M, 8 955 F, aucune date illisible. Les règles sont néanmoins implémentées, et **exercées** : `tests.demontrer qualite` injecte en bronze sept lignes fautives que la source ne contient pas — dates illisibles, sexe hors nomenclature, casse, pseudonyme vide côté patients et côté séjours — reconstruit silver dessus, et constate le sort de chacune avant de remettre l'entrepôt en état. Une règle qu'aucune donnée n'exerce ne prouve rien.

Ces deux contrôles n'ont pas la même issue, et c'est délibéré :

Anomalie	Issue	Pourquoi
Date d'admission illisible, ou date de sortie non vide et illisible	Écartée	Sans date fiable, ni la durée de séjour ni le rattachement d'un relevé ne tiennent. Le contrôle passe avant la cohérence temporelle : sinon la comparaison porterait sur un NULL et la ligne échapperait aux deux
Sexe hors nomenclature	Corrigée en 'inconnu', ligne conservée et signalée	Écarter le patient orphelinerait tous ses séjours, pour un attribut purement descriptif qui n'entre dans aucune clé

C'est ce que distingue la colonne **action** de la quarantaine. Sans elle, une correction fausserait l'équation de conservation ou resterait invisible.

Colonnes ajoutées par calcul ou par jointure :

Colonne	Origine
duree_jours	<code>dateDiff</code> admission → sortie. NULL si séjour en cours
est_en_cours	<code>discharge_ts</code> IS NULL
service_label	Jointure avec le référentiel des services
libelle	Jointure avec la nomenclature CIM-10
discharge_mode	Normalisation : '' → 'inconnu' — 683 séjours , tous en cours. Aucun séjour clos n'est privé de mode de sortie dans ce dépôt : la règle est en place, elle n'a ici à traiter que le cas légitime

Les 528 relevés postérieurs à la sortie du patient, et pourquoi 520 d'entre eux ne sont plus écartés. L'exploration (§ 6.3.c) avait repéré 528 relevés de monitoring dont l'horodatage tombe après `discharge_ts`. Le contrôle `releve_hors_sejour` ne s'évalue que pour un séjour **cohérent** — présent dans `silver.sejours` — car sur un séjour dont la sortie précède l'admission, on ne sait pas laquelle des deux dates est fautive, donc pas ce que serait « avant » ou « après ». Sur les 528 : **8** portent en outre des mesures hors plage physiologique et sont déjà écartées par `capteur_hors_plage`, comptées dans les 858 ci-dessus. Les **520** restants portent un séjour **incohérent** : ils échappent au contrôle temporel — non par exemption de règle, mais parce que la règle n'a pas de sens à leur appliquer — et sont **conservés** en `silver.monitoring` avec `sejour_cohérent = 0`. Aucun des 528 n'est donc écarté par `releve_hors_sejour` lui-même sur ce dépôt : c'est ce qui explique son compte à zéro dans le tableau des motifs ci-dessus.

Silver → Gold • modélisation dimensionnelle

Aucune ligne n'est perdue vers `gold.pilotage` : les trois faits reprennent exactement les volumes de silver. La transformation est structurelle.

Opération	Effet
Éclatement en faits et dimensions	4 tables silver → 3 faits + 3 dimensions, les dimensions d'abord
<code>age_au_sejour</code>	<code>toYear(date_admission) - dim_patient.birth_year</code> — dérivé contre la dimension , approximé à l'année
<code>tranche_age</code>	Calculée dans les faits, par tranches de 10 ans
<code>sexe</code>	Lu dans <code>dim_patient</code> , source de vérité unique
<code>alerte_fc</code> , <code>alerte_s-po2</code> , <code>alerte_temp</code> , <code>en_alerte</code>	Application des seuils paramétrés (<code>eds/config.py</code>) — règle métier, pas règle de validité
<code>est_urgence</code>	<code>admission_mode = 'urgence'</code>
<code>est_sejour_index</code>	Clos et patient non décédé — dénominateur de la réadmission
<code>suivi_readmission_30j</code>	Auto-jointure résolue une fois , à la construction — dénominateur AJUSTÉ (clos, non décédés)
<code>readmission_30j_brute</code>	Même auto-jointure, dénominateur BRUT (tous les séjours, décès compris) — chiffre de référence de l'intervenant (§ 6)
<code>sejour_coherent</code>	Recopié depuis <code>silver.diagnostics</code> / <code>silver.monitoring</code> — jamais un filtre en gold, cf. § 4.3
Dénormalisation	<code>patient_pseudo</code> , <code>service_code</code> , <code>date_admission</code> recopiés tels quels depuis <code>silver.diagnostics</code> / <code>silver.monitoring</code> (déjà enrichis en silver) ; <code>tranche_age</code> , <code>sexe</code> restent dérivés contre <code>dim_patient</code>
Axes temporels	<code>date_admission</code> pour les séjours, <code>date_mesure</code> pour les relevés — la mesure, jamais le dépôt

Puis, **dérivés de ces faits**, les huit indicateurs agrégés du pilotage :

Table	Grain	Lignes	Répond à
kpi_dms_service	service × mois	8	DMS — avec médiane, P90, max, et dms_heures en plus de dms_jours
kpi_urgences_jour	jour d'admission	28	activité des urgences — nb_passages_urgences (référence, service), nb_encore_presents , duree_moy_heures , nb_admissions_en_urgence (mode, complément)
kpi_readmission_service	service	8	réadmission à 30 jours — colonnes brutes ET ajustées côte à côte
kpi_alertes_jour	jour de mesure × service	59	relevés en alerte
kpi_occupation_jour	jour × service	224	« autre vue d'activité »
kpi_mortalite_service	service	8	idem
kpi_case_mix_service	service × pathologie	32	idem
kpi_origine_service	service × département	64	idem

Aucune information n'y est créée : ce sont des GROUP BY sur les faits, et l'égalité est vérifiée table par table. Chacune expose son effectif à côté de sa mesure, et kpi_readmission_service expose numérateur et dénominateur en plus du taux — de quoi recomposer l'indicateur sur un autre périmètre.

Vers gold_recherche , en revanche, la réduction reste volontaire, mais sa définition a changé (§ 5) :

Opération	Effet
nb_patients (référence)	Aucun filtre de type — tous les diagnostics, principal et associé : 12 720 lignes de faits
nb_patients_principal (complément)	Filtre est_principal : seul le motif d'hospitalisation est retenu — 6 797 diagnostics sur 12 720
Agrégation	12 720 lignes de faits (tous types) → 11 prévalences (coh_prevalence) ; 6 797 diagnostics principaux → 89 cohortes (coh_description)
HAVING >= 5 patients	Filtre appliqué à l'écriture, sur nb_patients : aucune cohorte sous seuil n'existe. Il retire ici 2 prévalences et 13 cohortes
Généralisation de l'âge	Tranches de 10 ans uniquement — birth_year absent
Suppression du pseudonyme	patient_pseudo n'est pas exposé

Bilan

Couche	Lignes	Détail	Ce qu'elle garantit
Lake	24 818 + Parquet	89 fichiers — 3 <code>patients.csv</code> , 28 <code>sejours.csv</code> , 28 <code>diagnostics.json</code> , 28 <code>monitoring.parquet</code> , 2 référentiels	Copie fidèle, sans aucune identité
Bronze	79 316	patients 18 000 · séjours 6 797 · diagnostics 12 720 · relevés 41 778 · référentiels 21	Typé, partitionné, traçable jusqu'au fichier d'origine
Silver	66 369	patients 6 000 · séjours 6 729 · diagnostics 12 720 · relevés 40 920	Nettoyé, cohérent, enrichi — et rien d'autre
Quarantaine	926	toute ligne écartée, avec son motif, son détail et sa provenance	Rien n'est perdu silencieusement
Gold pilotage	66 390	3 faits (6 729 + 12 720 + 40 920 = 60 369) · 3 dimensions (6 021)	Modèle dimensionnel interrogeable librement
Gold recherche	100	prévalences 11 · cohortes 89	Agrégats anonymisés, $k \geq 5$, aucun pseudonyme

4.5 Ce que le lake a le droit de contenir

La pseudonymisation protège ce qu'elle connaît. Elle ne dit rien de ce qu'un dépôt contiendra demain — et un entrepôt de santé reçoit des fichiers qu'il n'a pas écrits, dont le schéma peut changer sans préavis. Le lake applique donc une règle plus large que la seule pseudonymisation : **il ne contient que ce qui est déclaré**.

`config.COLONNES_LAKE` énumère, source par source et fichier par fichier, les colonnes autorisées à y entrer ; `eds.lake` projette chaque fichier sur cette déclaration avant d'écrire, quel que soit son format. C'est le principe qui gouverne déjà `patients` — sa transformation construit un dictionnaire neuf à quatre clés, où une colonne `email` ajoutée en amont ne peut pas entrer — appliqué à toutes les sources sans exception.

L'exception n'aurait d'ailleurs pas de sens. `actes`, `monitoring` et `diagnostics` ne portent pas d'identité aujourd'hui, mais elles sont liées au patient par `stay_id` : un `patient_id` ajouté au Parquet des actes, un nom de praticien ajouté au diagnostic, et l'identité traverserait le lake en clair sans que rien, en aval, ne la rattrape. Une source qui ne porte pas d'identité n'est pas une source qui n'en portera jamais.

Trois conséquences, toutes voulues :

- une colonne non déclarée n'atteint pas le lake, et **aucun code n'a besoin d'être modifié** pour cela ;
- un fichier dont le contenu n'est pas décrit — un `praticiens.csv` déposé demain dans les référentiels — n'est **pas copié du tout** : on ne sait pas ce qu'il contient, donc on ne sait pas qu'il est inoffensif ;
- le lake n'est pas une copie à l'octet, et c'est le prix assumé de la règle. Le CSV est réécrit ligne à ligne, le JSON objet par objet — imbrications comprises, sans quoi un champ ajouté au diagnostic lui-même passerait sous une projection de premier niveau — et le Parquet est reprojeté par DuckDB, qui ne lit que les colonnes retenues.

Les écarts sont journalisés dans les deux sens : une colonne apparue en amont est signalée **et** retirée ; une colonne déclarée qui disparaît de la source est signalée sans bloquer le dépôt du jour. C'est la même règle que pour les dates illisibles — détecter et tracer, jamais interrompre l'ingestion pour une anomalie de source.

La déclaration est aussi une documentation exécutable : elle dit en un coup d'œil ce que le lake est censé contenir, et `tests/test_config.py` vérifie qu'aucun nom de colonne identifiante n'y a été inscrit par mégarde.

5. Le modèle

Le § 6 du sujet décrit gold comme des « indicateurs par usage » — une table par indicateur. C'est le chemin de lecture le plus direct, et il correspond au besoin courant : la direction consulte une DMS, elle ne compose pas une requête. Mais pris seul, ce raccourci a un défaut rédhibitoire : **il fige les questions**. Croiser la durée de séjour par service *et* par tranche d'âge, ou la ventiler par pathologie, exige d'avoir anticipé la combinaison exacte et d'ajouter une table.

Nous avons donc fait les deux, dans cet ordre de dérivation :

Niveau	Contenu	Pour qui
Indicateurs agrégés	8 tables <code>kpi_*</code> : DMS, urgences, réadmission, alertes, occupation, mortalité, case-mix, origine géographique	La lecture courante — un indicateur, une table, aucune jointure
Modèle en étoile	3 faits, 3 dimensions conformes	L'analyse qu'aucune table figée ne couvre

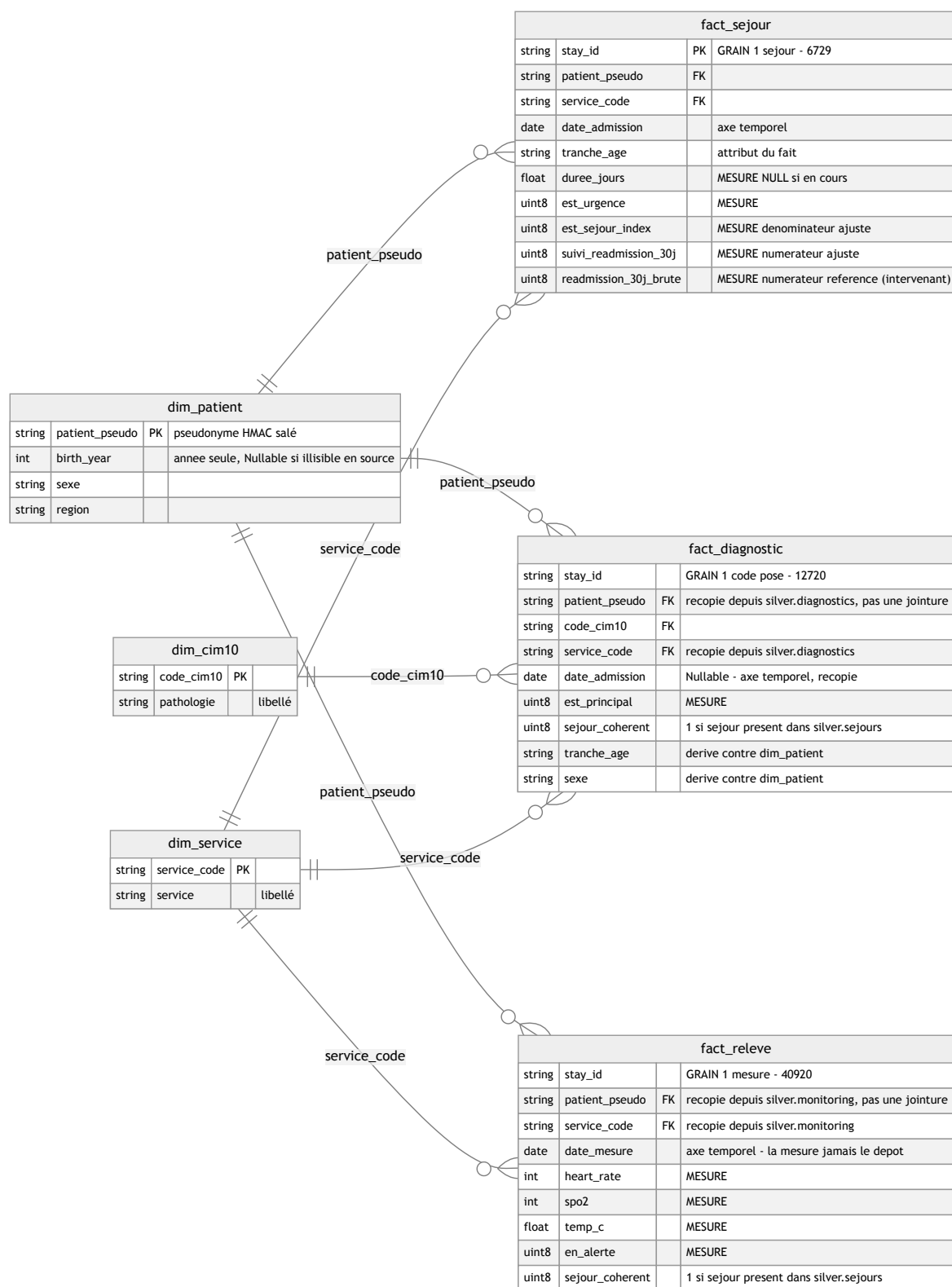
Les huit tables d'indicateurs sont **dérivées des faits**, jamais de silver. Un indicateur et le fait dont il sort doivent donner le même chiffre par construction — et `tests.verifier_indicateurs` confronte chaque table à son recalcul depuis les faits, en cardinalité comme en valeur. Sans ce contrôle, une table d'agrégat continuerait de servir un chiffre que plus rien ne fonde.

Chaque table porte son **effectif** à côté de sa mesure. Un taux sans son dénominateur ne s'interprète pas : 23,5 % d'alertes sur 34 relevés et 7,5 % sur 252 ne pèsent pas de la même façon — c'est exactement le cas qui se présente au dernier jour de la série, quand le dépôt s'amenuise.

L'étoile, sous les indicateurs : trois tables de faits, à trois grains distincts, partageant des dimensions conformes.

Fait	Grain	Volume
<code>fact_sejour</code>	un séjour	6 729
<code>fact_diagnostic</code>	un code posé lors d'un séjour	12 720
<code>fact_releve</code>	une mesure au chevet	40 920

Dimensions : `dim_patient`, `dim_service`, `dim_cim10`.



Pourquoi trois faits et non un seul. Les grains sont incompatibles : un séjour porte 1 à 3 diagnostics et 0 à n relevés. Les fusionner en une table unique multiplierait les lignes et fausserait toute somme — c'est le *fan trap* classique. Une DMS calculée sur une table jointe aux relevés compterait chaque séjour autant de fois qu'il a de mesures.

Deux dénormalisations assumées. `fact_diagnostic` porte le pseudonyme patient, son âge et son sexe, recopiés depuis le séjour : les questions de recherche portent sur des cohortes de patients par pathologie, et cette copie leur évite deux jointures systématiques. De même, le drapeau de réadmission est calculé **une fois** dans `fact_sejour` plutôt que laissé à l'outil de restitution : c'est une auto-jointure, hors de portée d'une requête d'analyse ordinaire.

Les faits se construisent sur les dimensions. Les trois dimensions sont écrites en premier ; les faits ensuite, et tout attribut de dimension dont un fait a besoin est lu **dans la dimension**, jamais re-dérivé depuis `silver`. C'est ce qui fait de l'étoile un modèle plutôt que trois tables qui se ressemblent, et c'est ce qui rend l'intégrité vérifiable : sept contrôles `fait → dimension` s'exécutent à chaque passage de `tests.verifier qualite`.

Le choix a une conséquence pratique : `fact_diagnostic` lit le sexe dans `dim_patient` et non dans `silver.patients`. L'attribut n'a qu'une source de vérité, et le jour où la dimension évolue — historisation, correction — les faits suivent sans qu'on ait à y penser.

Nuance à énoncer : `fact_diagnostic` et `fact_releve` ne rejoignent plus `silver.sejours`, ni `dim_service`, pour leurs attributs porteurs. Depuis la décision de l'intervenant sur la cohérence temporelle (§ 4.3), un diagnostic ou un relevé n'a plus besoin d'un `INNER JOIN silver.sejours` pour connaître son patient, son service et son admission : `silver.diagnostics` et `silver.monitoring` les portent déjà, enrichis **en silver**, contre `bronze.sejours`. Gold les recopie donc tels quels — `patient_pseudo`, `service_code`, `admission_ts` — sans les relire dans une dimension ni les rejoindre à `silver.sejours`. Le seul appariement de dimension qui subsiste au grain de l'événement est le `LEFT JOIN dim_patient` de `fact_diagnostic`, pour `age_au_sejour` et `sexe` ; `fact_releve` ne porte ni l'un ni l'autre et n'est joint à aucune dimension.

L'âge est un attribut du fait, dérivé contre la dimension. `dim_patient` ne porte pas d'âge : celui-ci dépend de la date de l'événement. Mais il ne se calcule pas non plus en `silver`, où on l'aurait figé trop tôt : `age_au_sejour` croise `dim_patient.birth_year` et l'axe `date_admission` du fait, au moment où le fait est construit. C'est la définition même d'un attribut dérivé de fait — ni une propriété de la personne, ni une donnée de la source.

La jointure est un `LEFT JOIN` volontaire. Un séjour dont le patient serait absent de la dimension doit rester dans le fait avec un âge `NULL`, jamais disparaître : c'est la même règle que partout ailleurs ici — on écarte explicitement, on ne perd pas silencieusement.

Ce que compte une cohorte : tous les types de diagnostic, le principal en complément. Chaque séjour porte un diagnostic principal et 0 à 2 associés — 6 797 principaux (un par séjour, y compris les 68 aux dates incohérentes, cf. § 4.3) contre 5 923 associés. `coh_prevalence.nb_patients` — la colonne de **référence**, celle qui porte désormais le filtre des petits effectifs — compte les patients distincts portant un code CIM-10 donné, **quel que soit son type** : motif d'hospitalisation ou comorbidité. C'est la définition retenue par l'intervenant, et c'est elle qui détermine ce qui est diffusé. Sur l'insuffisance cardiaque (I50), par exemple : 729 patients hospitalisés *pour* ce motif, **2 156** si l'on compte aussi ceux qui la portent en comorbidité — près du triple, et c'est ce second chiffre qui est publié au § 6.

`coh_prevalence.nb_patients_principal` reste exposée à côté, en complément : c'est l'**ancienne** définition de ce projet, restreinte au motif d'hospitalisation seul, toujours pertinente pour qui veut la prévalence au sens strict — mais elle ne détermine plus la diffusion.

La description de cohorte, elle, garde le grain du diagnostic principal. `coh_description` (âge × sexe) continue de filtrer `est_principal = 1` : croiser âge et sexe a un sens univoque sur le motif d'hospitalisation, moins sur une liste de comorbidités où un même patient apparaîtrait potentiellement dans plusieurs cellules d'une même pathologie.

Ce choix n'est pas neutre sur le chiffre obtenu : il doit donc accompagner l'indicateur partout où celui-ci est diffusé, et pas seulement figurer ici. Un indicateur dont on ne dit pas ce qu'il compte n'est pas exploitable — c'est le « justifier chaque chiffre » du § 4 du sujet. Le détail par type de diagnostic reste disponible en pilotage, où `fact_diagnostic` porte `est_principal`.

Le prix de ce choix. Les faits sont au grain de l'événement et portent le pseudonyme patient. Ils ne peuvent donc pas être exposés à la recherche, qui pourrait y reconstituer des cohortes sous le seuil. Ils vivent dans `gold_pilotage`, dont l'accès est restreint ; la recherche reçoit des agrégats dérivés de ces mêmes faits. C'est un arbitrage entre liberté d'analyse et exposition, tranché différemment selon le public.

6. Les indicateurs

Le § 4 du sujet demande de « justifier chaque chiffre ». Un tableau de bord affiche une valeur ; il ne dit pas sur quel dénominateur elle est calculée, ni si le périmètre annoncé est celui qui a servi. `tests.verifier indicateurs` restitue donc les six valeurs **et** les propriétés qui les fondent, dans la même exécution : si l'une tombe, le chiffre affiché à côté est faux, et le contrôle échoue avant qu'il ne soit diffusé.

Indicateur	Valeur sur le dépôt courant	Ce qui est vérifié en même temps
DMS par service	2,15 j (Urgences) à 9,05 j (Réanimation), sur 6 046 séjours clos — <code>dms_heures</code> désormais disponible (51,7 h à 217,1 h)	Les 8 services du référentiel sont représentés ; le dénominateur est exactement l'ensemble des séjours clos ; aucune durée manquante parmi eux ; <code>dms_heures</code> coïncide avec <code>dms_jours × 24</code> à $\pm 0,2$ h (double arrondi)
Passages aux urgences par jour	1 423 passages (<code>service_code = 'URGENCES'</code> , référence) sur 28 jours, 9 à 82 par jour ; jusqu'à 18 encore présents un jour donné ; durée moyenne 46,0 à 60,3 h. <code>nb_admissions_en_urgence</code> (mode d'admission, tous services) : 3 327, en complément	La série journalière somme au total ; l'axe est la date d' admission , jamais le jour de dépôt ; <code>nb_encore_present</code> ne compte que des séjours sans date de sortie ; les deux lectures — service et mode — coexistent sans se confondre
Réadmission à 30 jours	11,59 % — 780 réadmissions sur 6 729 séjours (BRUT, référence de l'intervenant) ; 12,81 % — 647 sur 5 051 séjours index (AJUSTÉ, décès exclus, complément documenté)	Le numérateur est inclus dans le dénominateur, dans les deux définitions ; aucune réadmission portée par un séjour hors dénominateur ; le brut confronte directement <code>sum(readmission_30j_brute)</code> à <code>count()</code> sur <code>fact_sejour</code>
Relevés en alerte	3 314 sur 40 920 (8,1 %) — FC 1 105 · SpO2 1 127 · T° 1 082, seuils FC < 50 ou > 100 bpm	<code>en_alerte</code> est bien la réunion des trois motifs ; les drapeaux sont recalculés depuis <code>eds/config.py</code> et doivent coïncider — ce qui prouve que le seuil configuré est celui qui a servi, et non une constante figée dans le SQL
Prévalence par pathologie	11 pathologies, <code>nb_patients</code> (référence, tous types) de 8 à 2 234 patients ; <code>nb_patients_principal</code> (complément, motif seul) de 8 à 729	L'agrégat reproduit exactement le recalcul depuis <code>fact_diagnostic</code> , sans filtre de type pour <code>nb_patients</code> ; <code>nb_sejours >= nb_patients</code> ; toute pathologie restituée existe dans la nomenclature
Distribution âge × sexe	89 cohortes, tranches 0-9 à 90-99	Aucune tranche <code>inconnu</code> ; sexe borné à la nomenclature ; la somme des cases d'une pathologie ne dépasse pas <code>nb_patients</code> (référence) de la prévalence ; aucune pathologie décrite hors prévalence

Les quatre vues du cinquième point. Le § 4 du sujet laisse une ligne ouverte après les quatre indicateurs nommés. Quatre vues la remplissent, chacune répondant à une question qu'aucune des quatre ne pose :

Vue	Valeur sur le dépôt courant	Ce qui est vérifié en même temps
Occupation	Cardiologie 352 présents en moyenne, pic à 473 ; 28 jours couverts	Chaque séjour compte une admission et une seule ; les sorties observées correspondent aux séjours clos dans la fenêtre ; jamais moins de présents que d'admis ; la série ne déborde pas la fenêtre d'observation
Mortalité	Pédiatrie 19,64 %, réanimation 18,2 % — chiffres non interprétables , voir ci-dessous	Le dénominateur est restreint aux séjours clos ; la table coïncide avec le fait
Case-mix	Pneumologie : 69,33 % de pneumopathies ; Urgences : 56,75 % d'infections urinaires	Les parts d'un même service somment à 100 % ; la table couvre tous les diagnostics principaux, y compris ceux des 68 séjours aux dates incohérentes
Origine géographique	8 départements de résidence par service, répartition assez plate (13,3 à 16,3 % pour le département le plus représenté d'un service)	Les parts d'un même service somment à 100 % ; <code>region_code</code> n'est retenu jusqu'en gold que parce que cette vue l'utilise — sinon la minimisation l'aurait exclu

La mortalité est publiée mais ne doit pas être lue. Sur les données fournies, les modes de sortie sont uniformément distribués : il en résulte 19,6 % de décès en pédiatrie, ce qu'aucun établissement réel ne présenterait. L'indicateur est juste — il coïncide avec le fait, son dénominateur est le bon — mais la donnée qui l'alimente ne l'est pas. Il figure pour la complétude de la vue métier, avec cette réserve attachée. C'est précisément le cas où « justifier chaque chiffre » consiste à dire qu'un chiffre ne vaut rien.

La dispersion des durées corrige un angle mort. `kpi_dms_service` porte désormais médiane, P90 et maximum à côté de la moyenne. En réanimation : moyenne 9,05 jours, médiane 8,21, **P90 à 18,03**. Publier la seule moyenne masquait la moitié de l'écart entre un séjour ordinaire et un séjour long — soit exactement ce qu'un service qui pilote ses lits a besoin de voir.

Chaque indicateur du pilotage existe en deux exemplaires, et c'est délibéré : la table agrégée, qui est le chemin de lecture, et le fait, qui est la source de vérité. Un cinquième contrôle par indicateur confronte les deux — même nombre de lignes, mêmes valeurs. C'est ce qui empêche une table d'agrégat de dériver en silence après un changement de règle appliqué d'un seul côté.

Le contrôle le plus utile est celui des seuils d'alerte. Ils sont présentés comme un paramètre d'exploitation, surchargeable par l'environnement sans toucher au SQL (§ 8.1). Cette affirmation serait invérifiable si le contrôle se contentait de compter les alertes : recalculer les drapeaux à partir de la valeur lue dans la configuration, et exiger l'égalité, est ce qui transforme la promesse en propriété. Changer `EDS_SEUIL_FC_BASSE` et relancer déplace les deux côtés de l'égalité ensemble ; laisser un seuil en dur dans le SQL les ferait diverger.

Un cinquième contrôle, d'une nature différente : la confrontation aux valeurs de référence de l'intervenant. Tout ce qui précède prouve des *propriétés* — équation de conservation, cohérence table ↔ fait, inclusion numérateur/dénominateur. Une propriété peut tenir alors que la *valeur* publiée n'est pas celle attendue : un dénominateur mal choisi peut rester cohérent avec lui-même sans être le bon. `tests.verifier_conformite` confronte donc directement l'entrepôt à des valeurs de référence fournies par l'intervenant — comptages exacts pour les effectifs, tolérance $\pm 0,1$ pour les moyennes (DMS, durées) — sur silver et sur les six indicateurs. C'est la seule section qui peut échouer sur une valeur juste alors que toutes les propriétés tiennent, et c'est voulu : c'est elle qui fait foi sur les définitions retenues dans ce rapport (§ 5, § 6). Le fichier de référence est **local, non versionné** (`eds-chu-sujet/`, exclu par `.gitignore`) : absent sur une machine qui ne l'a pas reçu, la section s'annonce ignorée plutôt que d'échouer, pour ne pas bloquer le reste de la suite.

Ce que ces contrôles ne remplacent pas. Ils établissent qu'un indicateur est calculé sur le périmètre annoncé, pas qu'il soit cliniquement pertinent. Les réserves du § 8.1 — historique de 28 jours pour un indicateur à 30, monitoring limité à deux services, données synthétiques — restent entières et doivent accompagner chaque chiffre là où il est diffusé.

7. Les visualisations

7.1 À quelle question répond chaque carte

Un tableau de bord n'est pas une collection de graphiques disponibles : c'est une suite de questions qu'un métier se pose. Trente-et-une questions, réparties en trois vues qui ne partagent aucune donnée.

Carte	La question posée	Pour qui
Séjours · DMS · réadmission · taux d'alerte	les quatre nombres qu'on cite en réunion de direction	direction
DMS par service	quels services gardent les patients le plus longtemps ?	direction
Dispersion des durées	la moyenne cache-t-elle une queue longue ?	cadres de santé
Réadmission par service	d'où les patients repartent-ils trop tôt ?	cadres de santé
Passages aux urgences par jour	la charge des urgences suit-elle un rythme ?	cadre des urgences
Occupation par jour et par service	l'hôpital se remplit-il ou se vide-t-il ?	direction
Relevés en alerte par jour	la surveillance signale-t-elle plus certains jours ?	cadres de santé
Case-mix par service	quelles pathologies chaque service traite-t-il réellement ?	direction
Mortalité par service	où les séjours se terminent-ils par un décès ?	direction
Origine géographique	d'où viennent les patients ?	direction

La recherche pose d'autres questions — *quelle est la prévalence de chaque pathologie, comment se répartissent les patients par âge et par sexe* — et aucune ne demande un service ni une journée. C'est pourquoi son tableau de bord n'en montre aucun : ce n'est pas une restriction imposée de l'extérieur, c'est que la question ne se pose pas.

7.2 Pourquoi ces formes, et pas d'autres

Le choix n'est pas esthétique : il découle de ce que la donnée **est**. Cinq formes, 31 cartes, et une règle par forme.

Ce qu'on montre	Forme retenue	Emplois	Pourquoi
Un nombre isolé	scalaire	11	rien à comparer — un graphique le noierait
Des catégories à classer	barres horizontales , triées	9	le libellé se lit sans incliner la tête, et le tri fait le classement
Une série journalière	courbe	3	la continuité du temps est réelle, la ligne la montre
Un stock décomposé	aires empilées	1	l'occupation se répartit entre services, et la somme a un sens
Une liste à lire	table	7	un case-mix ou une répartition d'actes se lit ligne à ligne, pas en longueurs comparées

Les barres sont horizontales, jamais verticales. Huit services aux noms longs — « Réanimation », « Pneumologie » — obligerait à incliner des étiquettes verticales, ou à les tronquer. Et un classement se lit de haut en bas, comme un podium.

Une objection honnête sur les courbes. Un flux — des passages aux urgences — se compte par jour et ne se prolonge pas d'un jour sur l'autre : on peut soutenir qu'il appelle des barres verticales, et que la courbe suggère à tort une continuité. Nous avons retenu la courbe pour une raison de lecture : ces séries servent à repérer un **rythme** — le creux du week-end, un pic — et l'œil suit une ligne mieux qu'il ne compare trente barres. La forme est donc un compromis assumé entre la nature de la donnée et l'usage qu'on en fait. L'occupation, elle, est bien un stock — un patient présent un jour l'est encore le lendemain — et reçoit une aire.

Une même entité garde sa couleur d'une carte à l'autre. C'est ce qui permet de suivre un service d'un graphique au suivant sans relire la légende.

7.3 Les réserves sont dans les tableaux de bord, pas seulement ici

Trois réserves sont écrites **dans les cartes elles-mêmes**, à côté des courbes qu'elles nuancent : le monitoring ne concerne que deux services sur huit, la courbe d'occupation s'arrête au dernier dépôt et non à la date du jour, et la Neurologie manque au graphe de densité par lit faute de capacité connue.

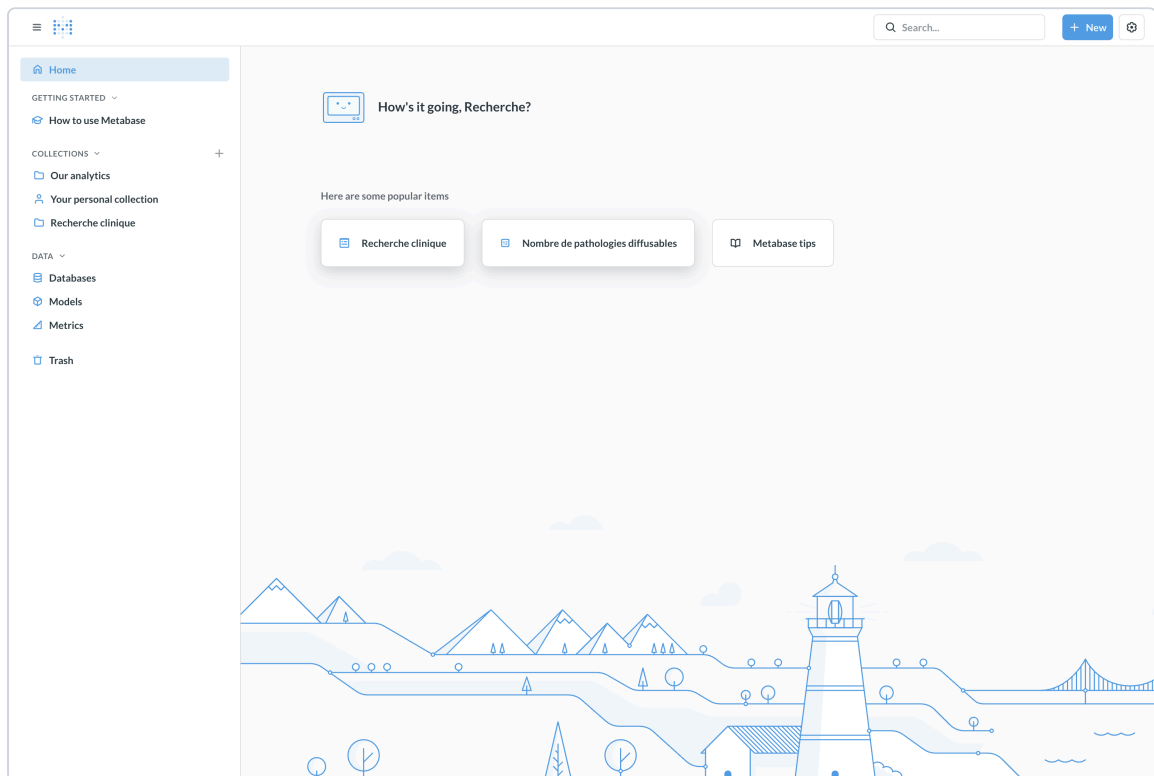
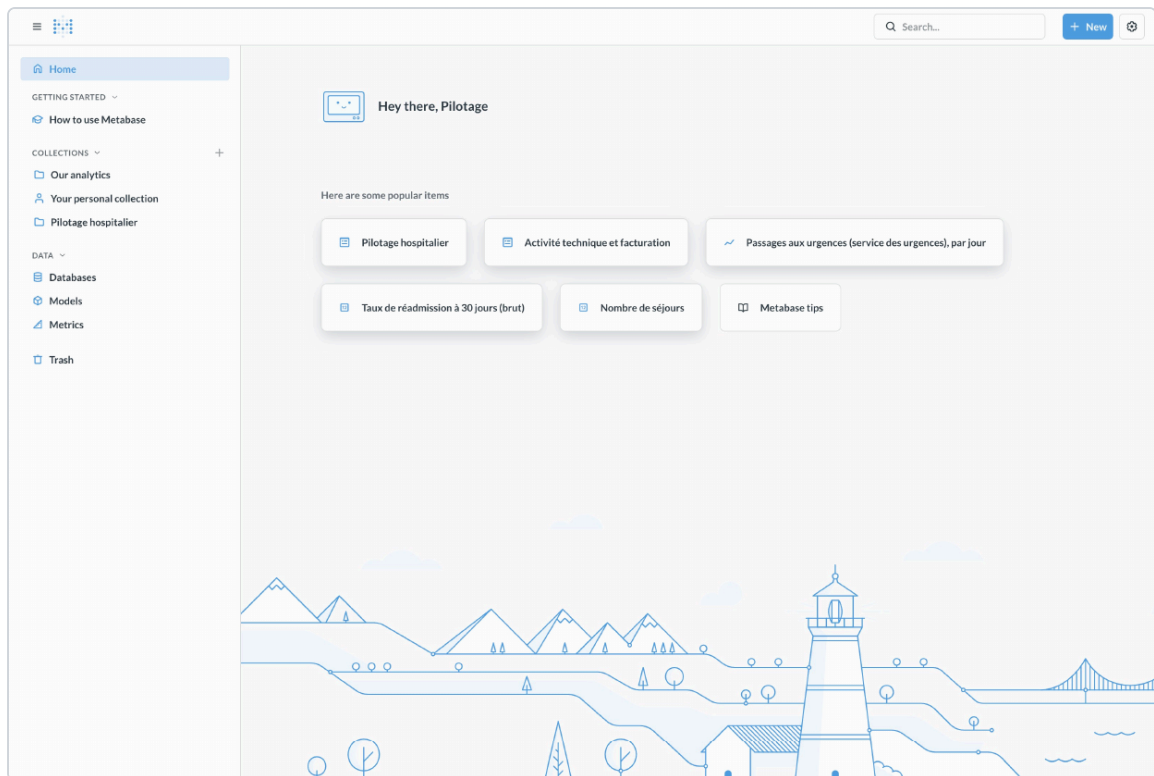
Les reléguer dans ce rapport reviendrait à supposer qu'un lecteur de tableau de bord l'aura ouvert. Un chiffre qui appelle une réserve doit la porter là où il s'affiche.

7.4 Pourquoi la restitution n'affaiblit pas le cloisonnement

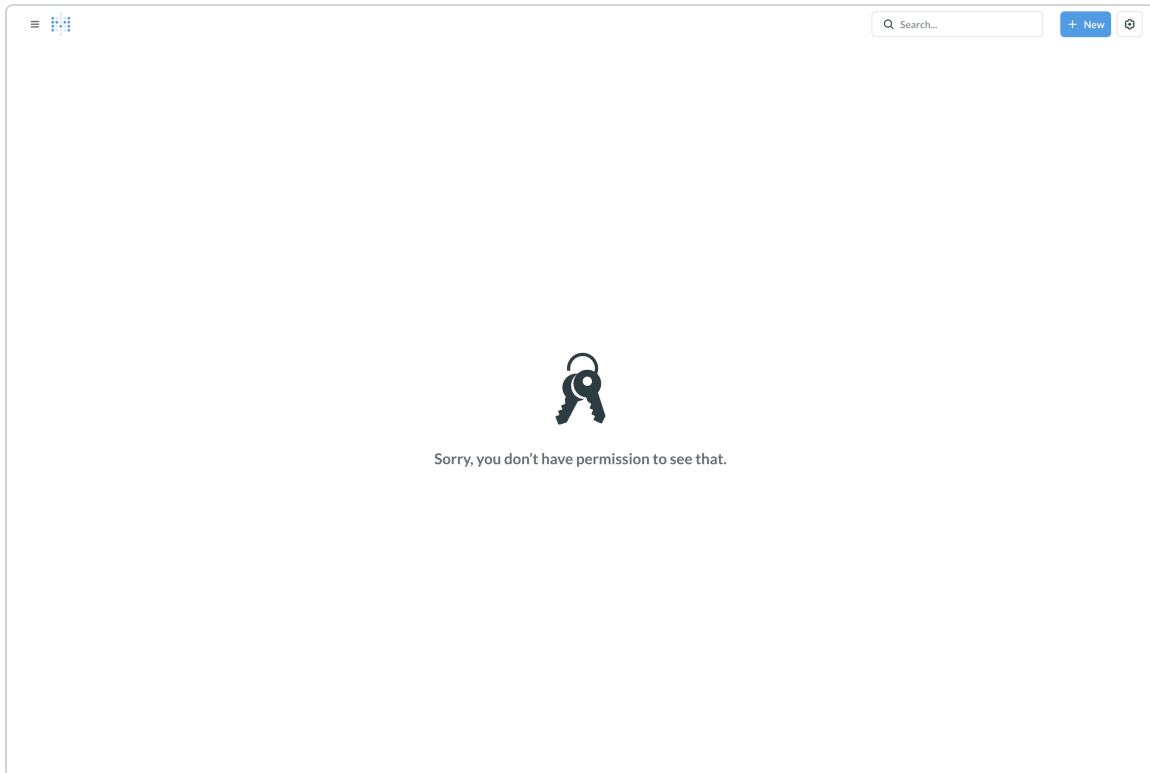
Le § 1 du sujet demande une interface — au moins deux tableaux de bord, pilotage et recherche, avec démonstration du cloisonnement des droits — et conseille Metabase. `eds/restitution.py` la provisionne intégralement par son API REST : deux connexions ClickHouse, deux groupes, trois comptes applicatifs, un graphe de permissions et les deux tableaux de bord, jamais posés à la souris. Quatre choix méritent d'être justifiés séparément.

Metabase, sans code applicatif. Le sujet le conseille explicitement, et il correspond exactement au besoin : deux publics qui consultent des indicateurs déjà agrégés, jamais une équipe qui développerait une interface sur mesure. Aucune ligne de code métier ne s'ajoute au projet — le module qui le provisionne parle HTTP, pas une API de rendu — ce qui laisse la totalité de l'effort d'ingénierie sur l'entrepôt, où se joue réellement le cloisonnement.

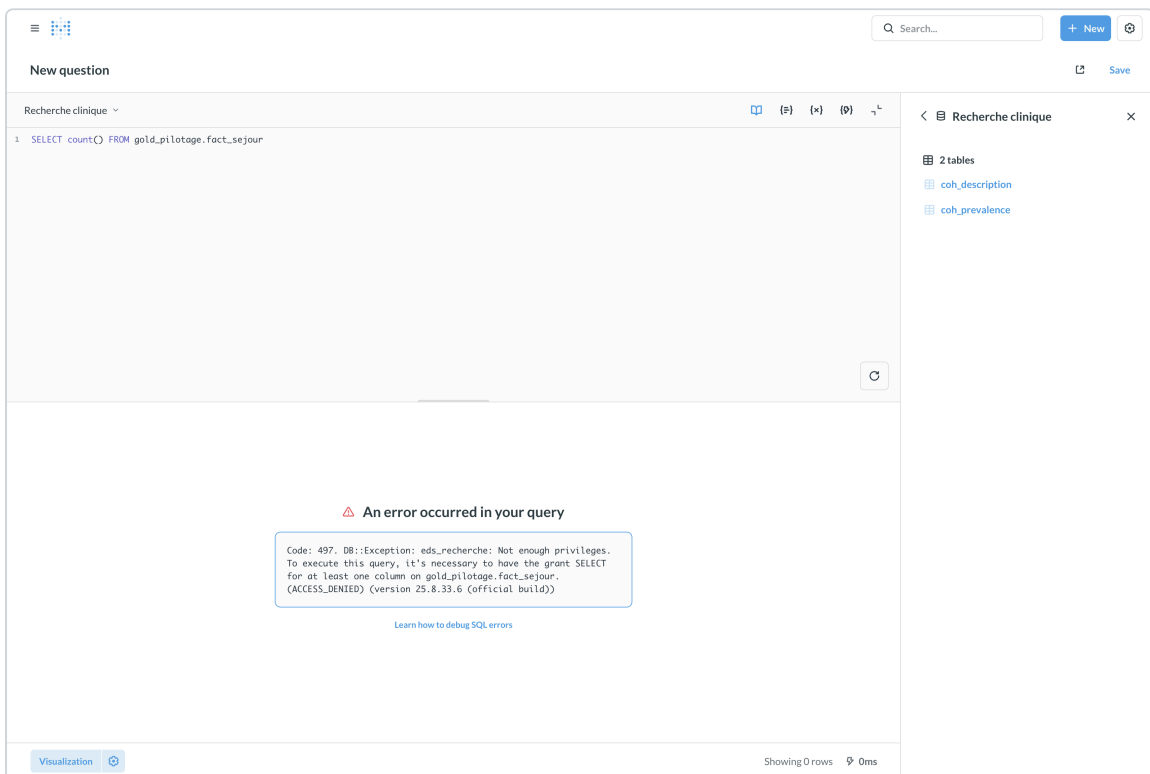
Chaque connexion Metabase s'authentifie avec le compte ClickHouse borné de son usage — `eds_pilotage` ou `eds_recherche` — jamais avec un compte unique. L'alternative la plus simple à écrire aurait été une seule connexion administrateur, puis un filtrage applicatif dans Metabase (permissions par groupe, par collection). Elle est écartée pour la même raison que le § 3.4 écarte un filtre applicatif au niveau SQL : elle serait contournable par quiconque atteint l'API Metabase avec des droits suffisants, ou par une mauvaise manipulation du graphe de permissions. En branchant chaque connexion sur le compte déjà borné par `sql/50_droits.sql`, le refus qu'un utilisateur de Pilotage rencontre sur la base Recherche n'est pas un réglage de plus à maintenir en cohérence avec ClickHouse : c'est le **même** refus, prononcé par le **même** moteur, qu'on interroge en SQL direct ou depuis un tableau de bord. `tests.demontrer_restitution` le prouve en forçant, avec le compte **administrateur** Metabase, une requête native sur la mauvaise base à travers chacune des deux connexions : le refus renvoyé porte le message ClickHouse lui-même (« Not enough privileges... ACCESS_DENIED »), pas un message Metabase — la preuve que même le compte le plus privilégié de l'interface ne franchit pas la borne.



Les deux vues vont par paire, et c'est la paire qui prouve quelque chose : le compte nommé en tête de page ne voit, dans sa barre latérale, que sa propre collection. Une seule des deux captures laisserait penser que la collection absente n'est peut-être pas épinglée.



Forcer l'adresse du tableau de bord étranger ne contourne rien. Ici c'est Metabase qui refuse — la capture suivante montre que le moteur refuse aussi, indépendamment.



La démonstration décisive. Le compte recherche compose une question SQL native sur `gold_pilotage.fact_sejour` : le message affiché est celui de ClickHouse, « eds_recherche: Not enough privileges... ACCESS_DENIED », pas un message Metabase. Le panneau de droite ajoute une confirmation : la connexion Recherche n'expose que 2 tables, les deux tables d'agrégats — le reste de l'entrepôt n'existe pas pour elle.

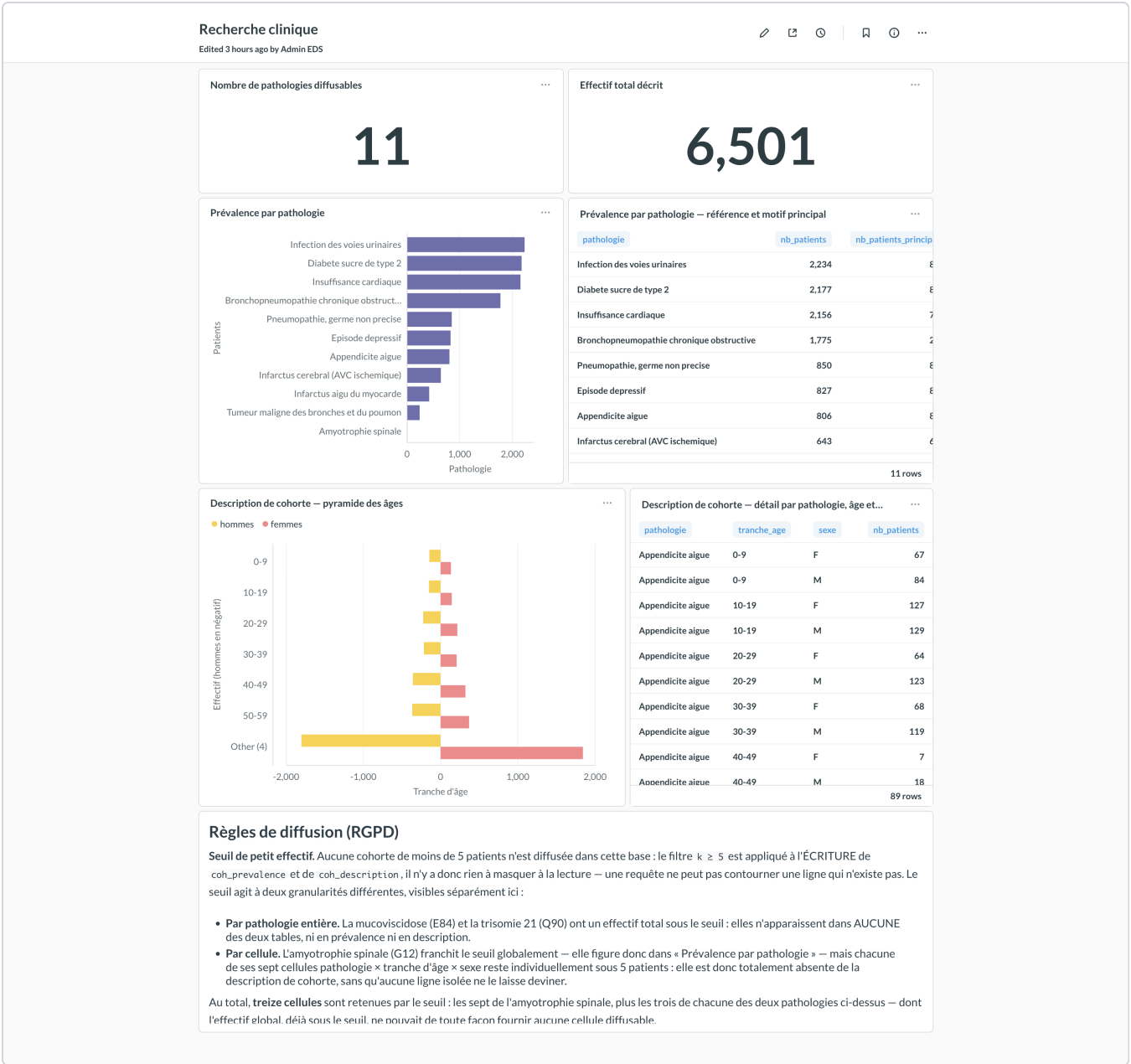
Chaque question du tableau de bord est du SQL natif (`dataset_query.type = "native"`), jamais le query-builder graphique de Metabase. Un query-builder recompose une requête à partir de clics, avec un SQL généré qu'il faut relire pour être sûr qu'il interroge la bonne table et le bon filtre. Une question SQL native se lit directement dans `eds/restitution.py`, se rejoue à la main dans la console ClickHouse pour vérifier le chiffre affiché, et se provisionne par API sans ambiguïté de traduction. Le chiffre qu'un tableau de bord affiche est donc, à la lettre, celui que renvoie la table gold — jamais une agrégation recomposée côté Metabase à partir d'une lecture plus fine.

Ce que chaque tableau de bord donne à voir. Vingt-deux questions et quatre cartes de texte, réparties en deux vues qui ne partagent aucune donnée.

« Pilotage hospitalier » — 16 questions	Forme
Nombre de séjours · DMS globale · réadmission à 30 j · taux d'alerte · date de dernière construction	cinq compteurs, en tête
DMS par service	barres, plus une table de dispersion (médiane, P90, maximum)
Passages aux urgences (service) et admissions en urgence (mode d'admission)	deux séries journalières distinctes , jamais confondues
Réadmission à 30 jours par service	barres, plus une table exposant numérateur et dénominateur
Relevés de constantes en alerte	série journalière
Occupation par jour et par service	aires empilées
Case-mix et origine géographique par service	deux tables de parts
Mortalité hospitalière par service	barres

« Recherche clinique » — 6 questions	Forme
Pathologies diffusables · effectif total décrit	deux compteurs
Prévalence par pathologie	barres, plus une table opposant l'effectif de référence à celui du seul motif d'hospitalisation
Description de cohorte, âge × sexe	pyramide des âges, plus la table détaillée par pathologie

« Pilotage hospitalier » — les quatre indicateurs nommés du § 4 du sujet et les vues complémentaires. Les réserves sont écrites dans le tableau de bord lui-même, à côté des courbes qu'elles nuancent, et non reléguées dans ce rapport.



« Recherche clinique » — les mêmes données sources, une lecture entièrement différente : des agrégats filtrés au seuil de 5 patients, aucun pseudonyme, l'âge en tranches de dix ans. Les deux vues ne partagent aucune table.

Trois réserves sont écrites dans les tableaux de bord eux-mêmes, pas seulement dans ce rapport : le monitoring ne concerne que deux services, la mortalité n'est pas cliniquement interprétable sur des données synthétiques (§ 6), et la base recherche ne diffuse aucune cohorte de moins de cinq patients. Un indicateur voyage sans son dossier : la réserve doit être là où le chiffre est lu, sans quoi elle ne protège personne.

Le provisionnement se fait par API, pas à la souris. Le reste de l'entrepôt (lake → bronze → silver → gold, droits ClickHouse) est entièrement scripté et rejouable ; une restitution posée manuellement dans l'interface aurait rompu cette propriété et ne serait démontrable qu'en capture d'écran. eds.restitution pilote Metabase exactement comme eds.run pilote ClickHouse : idempotent (recherche par nom avant toute création, mise en page d'un tableau de bord intégralement reposée à chaque passage plutôt qu'accumulée), et vérifié (chaque question est interrogée après coup via /api/card/:id/query avant de rendre la main — un tableau de bord silencieusement cassé ne peut pas sortir du provisionnement). Une relance ne change ni le nombre d'objets Metabase ni leur contenu.

8. Limites et recommandations

8.1 Limites

Douze limites, de quatre natures différentes. Le classement ci-dessous sert à les lire ; le détail suit dans l'ordre où chacune se rattache à ce qui précède.

Nature	Limites concernées
Ce que l'entrepôt ne montre pas — le périmètre des données le borne	historique de 28 jours et troncature de la réadmission · monitoring sur 2 services sur 8 · montée artificielle de la courbe d'occupation
Ce qui demande une relecture médicale — choisi par des informaticiens	seuils d'alerte, qui ne sont pas une norme clinique · âge approximé à l'année
Ce qui appartient aux données, pas au pipeline	données synthétiques · seuil des petits effectifs, qui protège chaque cellule mais pas leur différence
Ce qu'une mise en production changerait	recalcul intégral non tenable à l'échelle · sources non versionnées · grain de l'événement exposé au pilotage · périmètre de la restitution

L'historique de 28 jours tronque encore le taux de réadmission. La fenêtre d'observation reste plus courte que celle de l'indicateur — les délais constatés vont de 3 à 23 jours, et un patient sorti le 28 août ne peut pas être suivi jusqu'au 27 septembre. Le taux de **référence** publié au § 4 du sujet et au § 6 est le **BRUT** — 11,59 % (780 réadmissions sur 6 729 séjours, décès compris) : sa définition fait foi (valeurs fournies par l'intervenant), et c'est lui qui subit pleinement la troncature de la fenêtre. Le taux **AJUSTÉ** — 12,81 % (647 sur 5 051 séjours index, décès exclus) — reste exposé en complément documenté : un patient réadmis après un décès enregistré est une incohérence de saisie, mais la référence ne l'exclut pas, d'où deux chiffres plutôt qu'un arbitrage silencieux. Dans les deux cas c'est un **plancher** : l'écart n'est plus que de deux jours, ce qui le rend exploitable comme ordre de grandeur, mais il faut 90 jours pour une tendance.

Le monitoring ne couvre que 12,8 % des séjours, sur deux services (Cardiologie 41,9 %, Réanimation 40,7 % — mesuré sur `silver.sejours`). Les six autres n'ont **aucun** relevé. Les indicateurs de constantes sont explicitement restreints à ce périmètre et ne sont pas extrapolables.

Les seuils d'alerte ne sont pas fournis par le CHU au sens d'une norme clinique, et il n'en existe aucun qui soit réglementaire. Les moniteurs de chevet sortent d'usine avec des valeurs par défaut — chez l'adulte, alarme basse de fréquence cardiaque autour de 50 bpm en avertissement et 40 en critique — que chaque service, puis chaque soignant, sont censés adapter au patient : bêta-bloquants, sportif, nouveau-né. Ce n'est donc pas une propriété de la donnée mais un **paramètre d'exploitation**. Les valeurs par défaut retenues sont désormais celles **fixées par l'intervenant** — FC hors [50, 100] bpm, SpO2 < 92 %, température > 38,5 °C — pour reproduire l'indicateur de référence ; le mécanisme de surcharge est inchangé : elles restent externalisées dans `eds/config.py` et surchargeables par l'environnement (`EDS_SEUIL_FC_BASSE=45` , exemple qui reste distinct des deux bornes par défaut), et doivent être validées médicalement avant tout usage clinique. Avec ces seuils, le taux d'alerte est plat entre les deux services équipés — 8,1 % en Cardiologie, 8,0 % en Réanimation — ce qu'aucune donnée clinique réelle ne présenterait (cf. « données synthétiques » ci-dessous). Le pas suivant, hors périmètre ici faute de source pour l'alimenter, serait un référentiel de seuils **par service** — la réanimation et la médecine n'ont pas les mêmes presets.

Le recalcul intégral de silver et gold ne passera pas à l'échelle. Il est instantané sur 42 000 relevés ; il deviendra le goulot d'étranglement au-delà de quelques dizaines de millions de lignes.

L'âge est approximé à l'année, conséquence directe de la généralisation RGPD. Erreur maximale : un an.

Les données fournies restent synthétiques. Elles portent désormais des écarts plausibles entre services — DMS de 2,15 jours aux Urgences à 9,05 en Réanimation, une gradation cohérente avec la nature des prises en charge — mais elles sont générées, et le taux d'alerte reste, lui, plat (8,1 % en Cardiologie comme en Réanimation). La plateforme restitue fidèlement ce que contiennent les sources ; aucune conclusion médicale ne peut en être tirée.

Les fichiers sources ne sont pas versionnés. Un clone du dépôt ne suffit donc pas à exécuter le pipeline : il faut y placer le dépôt du CHU. C'est un choix délibéré — faire entrer des identités de patients dans un historique Git, d'où elles ne peuvent plus être retirées, serait contradictoire avec l'objet même de ce projet.

Les faits exposent le grain de l'événement au compte pilotage. Le sujet décrit la couche gold comme des « indicateurs par usage » : les huit tables `kpi_*` répondent à cette attente et constituent le chemin de lecture normal. Mais le compte de pilotage conserve aussi l'accès à dix-sept colonnes des faits, si bien qu'il peut composer des croisements qu'aucune table figée ne couvre — au prix d'un accès au grain du séjour.

Cet écart est délibéré : il achète une liberté d'analyse que les indicateurs figés ne permettent pas — croiser la durée de séjour par service *et* par tranche d'âge ne demande aucune table supplémentaire. Les données restent pseudonymisées et l'accès nominatif et restreint.

Et il est désormais réversible sans rien perdre. Depuis que les indicateurs existent en tables, borner `eds_pilotage` aux seules `kpi_*` suffirait à lui retirer tout accès au grain de l'événement : les six indicateurs demandés continueraient de fonctionner, seule la liberté de croisement disparaîtrait. C'est la décision à acter avec le DPO — elle ne coûte plus qu'un `REVOKE`.

Le seuil des petits effectifs protège chaque cellule, pas leur différence. `coh_prevalence` publie l'effectif d'une pathologie et `coh_description` ses cellules âge × sexe au diagnostic principal, chacune filtrée à $k \geq 5$. Les deux tables étant lisibles par le même compte, leur **différence** est calculable : `nb_patients_principal` moins la somme des cellules publiées donne le total des cellules supprimées.

Un cas réel l'illustre sur ce dépôt. L'amyotrophie spinale (G12) est diffusée en prévalence — 8 patients, au-dessus du seuil — mais aucune de ses 7 cellules âge × sexe n'atteint 5 : `coh_description` n'en publie donc aucune, et la soustraction $8 - 0$ révèle que 8 patients se répartissent en cellules toutes inférieures à 5. La borne n'est pas franchie — aucune cellule n'est identifiée, et le total était déjà public — mais la mécanique est là : si **une seule** cellule d'une pathologie était supprimée, la même soustraction en donnerait l'effectif exact. Ce cas-là ne se présente pas ici (les autres pathologies n'ont aucune cellule supprimée), et c'est un hasard des données, pas une garantie du modèle.

Deux réponses possibles, à arbitrer avec le DPO : la **suppression secondaire** — masquer une deuxième cellule dès qu'une première l'est — ou le retrait de `nb_patients_principal` de la base recherche, qui supprime le terme de comparaison. La seconde suffit ici, puisque `nb_patients` (tous types) et `coh_description` (principal seul) ne portent pas sur le même périmètre et ne se soustraient donc pas.

La courbe d'occupation monte artificiellement sur ses premiers jours. Aucun séjour antérieur au 1er août n'est connu : `kpi_occupation_jour` ne compte, ce jour-là, que les 243 séjours admis le jour même, alors qu'un hôpital réel héberge déjà des patients entrés la veille. La montée des premiers jours — 243, puis 463, puis 666 — est donc un artefact d'observation, pas une variation d'activité. La lecture ne devient représentative qu'une fois la fenêtre plus longue que la durée de séjour maximale.

La restitution règle le cloisonnement, pas ce qui l'entoure. Trois limites propres à Metabase, distinctes de celles de l'entrepôt ci-dessus :

- **La base applicative de Metabase (H2, embarquée dans le conteneur) n'est pas adaptée à une mise en production.** Elle convient à une démonstration — un seul fichier, aucune dépendance externe — mais ne supporte ni sauvegarde à chaud, ni accès concurrent en écriture au-delà d'un usage modeste, ni réplication. C'est un choix délibérément temporaire, cohérent avec le périmètre de ce projet.
- **Ni SSO, ni journalisation des accès aux tableaux de bord.** Les trois comptes Metabase (§ 7.4) s'authentifient par mot de passe local, indépendant de tout annuaire du CHU ; et si Metabase consigne bien l'exécution de chaque question dans sa propre base H2, ce journal n'est ni exporté vers `ops.executions`, ni exploité par ce projet — qui a consulté quel tableau de bord, et quand, ne se répond donc pas aujourd'hui, alors que c'est précisément la question que poserait un audit d'accès à des données de santé.
- **Les réserves cliniques reposent sur des cartes de texte, qu'un utilisateur peut ignorer.** La réserve méthodologique sur la mortalité et la mention des deux seuls services équipés de monitoring (§ 6) sont des cartes texte posées à côté des graphiques correspondants, pas une contrainte qui empêcherait de lire le chiffre sans elles. Rien dans Metabase ne force leur lecture, ni n'interdit d'exporter le graphique seul, sans son avertissement.

8.2 Recommandations

Priorité	Recommandation
Haute	Étendre l'historique à 90 jours minimum avant d'exploiter le taux de réadmission comme une tendance. Les 28 jours disponibles en font un ordre de grandeur, pas une mesure : la réserve doit rester énoncée partout où l'indicateur est diffusé.
Haute	Faire valider les seuils d'alerte par le corps médical. Ils sont désormais configurables (<code>eds/config.py</code> , surcharge par <code>EDS_SEUIL_*</code>) et non plus codés dans le SQL ; reste à les faire arbitrer, puis à les décliner par service.
Haute	Soumettre la stratégie de pseudonymisation au DPO, en particulier la conservation du triplet (année, sexe, région) en base pilotage. La région a désormais un usage nommé — la vue d'origine géographique par service — qui fonde sa conservation au titre de la minimisation ; c'est cet usage, et lui seul, que le DPO a à valider.
Haute	Remplacer la base applicative H2 de Metabase par PostgreSQL avant toute mise en service — sauvegarde à chaud, accès concurrent, réplication : ce que H2 ne couvre pas (§ 8.1).
Moyenne	Brancher Metabase sur le SSO / annuaire du CHU plutôt que sur des comptes locaux, et mettre en place un journal des consultations de tableau de bord — qui a vu quel indicateur, et quand — exigible pour un audit d'accès à des données de santé.
Moyenne	Brancher <code>EDS_ALERTE_CMD</code> sur la supervision déjà en place au CHU. Le mécanisme d'alerte existe et est testé (§ 9) ; le canal, lui, est une décision d'exploitation qui n'a pas à vivre dans le dépôt. À traiter avec la règle de détection d'un dépôt qui cesse — la limite qui subsiste.
Moyenne	Formaliser la gestion du sel : conservation en coffre, procédure de rotation, et conséquence assumée — sa perte rend tout rapprochement avec la source définitivement impossible.
Moyenne	Passer silver et gold en construction incrémentale si le volume dépasse quelques dizaines de millions de lignes.
Moyenne	Faire confirmer par le CHU que <code>discharge_mode</code> vide signifie toujours « séjour en cours ». C'est le cas sur ce dépôt — aucun séjour clos n'en est privé — mais la normalisation en <code>'inconnu'</code> masquerait une anomalie de saisie si la règle changeait à l'amont.
Basse	Étendre l'équipement de monitoring aux autres services, ou acter que cet indicateur restera limité à deux services.
Basse	Mettre en place une purge automatique selon les durées de conservation, actuellement non définies par le CHU.
Basse	Instaurer une revue périodique des habilitations Metabase et ClickHouse — qui appartient à quel groupe, quel compte a quel mot de passe — pour qu'un départ ou un changement de poste se traduise sans délai par un accès retiré.

Partie 2 — L'automatisation

Ce que le sujet nomme sa seconde partie : la collecte et la transformation planifiées, la gestion des erreurs, la journalisation, la traçabilité — et la conduite au quotidien, jusqu'à la reprise après incident.

9. Ce qui déclenche le pipeline

```
10 3 * * * cd $EDS_HOME && .venv/bin/python -m eds.supervision >> logs/cron.log 2>&1
```

03h10, et pas un autre horaire. Le choix est conventionnel : après une nuit de dépôt supposée terminée, avant la prise de poste du matin — de sorte qu'un incident se découvre en arrivant, jamais en pleine journée de soin. Il n'est pas ajusté sur une fenêtre de dépôt observée : le dépôt fourni date chaque jour au jour près (`_jour_depot` , type `Date` en bronze — § 3.5), sans horodatage intra-journalier, et les fichiers du dépôt courant portent tous la même date de fichier, preuve d'une extraction unique plutôt que de dépôts échelonnés dans la nuit. Rien dans ce dépôt ne permet donc de mesurer une fenêtre réelle ; 03h10 reste une hypothèse à confirmer en production, pas une valeur calée sur une observation. L'heure est un paramètre du fichier `ops/crontab.example` , pas une constante du code : la déplacer ne touche à rien d'autre.

cron , et pas un ordonnanceur applicatif. Airflow, Dagster ou un service managé apportent un graphe de dépendances entre tâches, des reprises automatiques et une interface de supervision. Aucun des trois n'a d'usage ici : le pipeline est **une seule tâche**, appelée une fois par jour, dont l'orchestration interne (`schema` → `lake` → `bronze` , par jour → `referentiels` → `silver` → `gold` → `droits` — sept des huit noms d'étape du § 11, le huitième, `restitution` , relevant d' `eds.restitution`) est déjà écrite en Python dans `eds/run.py` — le graphe de dépendances existe, mais à l'intérieur d'un seul processus, pas entre plusieurs tâches cron. Ajouter un ordonnanceur ne changerait aucune de ces dépendances ; il ajouterait une base de métadonnées et un serveur web à faire tourner pour planifier un appel quotidien.

Le coût a été mesuré plutôt que supposé : résolue contre l'environnement du projet, l'installation de `dagster` et de son interface web tire **64 paquets transitifs** — `grpcio`, `protobuf`, `pydantic`, `SQLAlchemy`, `alembic`, `uvicorn`. Le pipeline en compte deux (§ 2.3), et ce compte est un argument, pas un hasard. S'y ajouterait un démon à maintenir en vie en permanence, dont la mort n'empêcherait rien de se déclencher — silencieusement. C'est un choix à revoir le jour où plusieurs pipelines indépendants doivent se coordonner — pas avant. Ce que l'ordonnanceur aurait apporté d'utile ici — verrou, relance, alerte — tient dans `eds/supervision.py` , 229 lignes sans dépendance, couvertes par 24 tests hors ligne : 18 sur le superviseur lui-même, un faux pipeline injecté à la place d' `eds.run` , et 6 sur ses réglages, refusés à la frontière comme les seuils (§ 4.1).

Ce que cron ne garantit pas, et ce qu'on met en face. `cron` déclenche, et rien d'autre. La ligne de `crontab` n'appelle donc pas `eds.run` directement mais `eds.supervision` , qui l'encadre sans rien changer à son interface — les options sont transmises telles quelles, et un lancement manuel de `eds.run` reste possible et inchangé.

<code>cron</code> ne fait pas	Ce que <code>eds.supervision</code> met en face
Empêcher deux exécutions de se recouvrir	Un verrou (<code>logs/.verrou</code>), créé en <code>0_EXCL</code> — donc atomique : deux exécutions lancées à la même seconde ne peuvent pas conclure toutes les deux qu'elles l'ont pris. Le cas réel n'est pas théorique : un <code>--tout</code> lancé à la main à 03h09 et le cron de 03h10 écriraient bronze en même temps. Le verrou désigne un PID, et celui d'un processus mort est repris : une machine redémarrée en pleine nuit ne bloque pas les suivantes
Relancer après un échec	3 tentatives espacées de 10 min sur un échec <i>inattendu</i> (code 2 : ClickHouse arrêté, disque plein — une cause qui peut disparaître d'elle-même). Aucune sur une erreur <i>métier</i> (code 1 : jour absent, SQL invalide) : la même classification qu'au § 10, appliquée un cran plus haut. Les deux bornes sont des paramètres d'exploitation (<code>EDS_RELANCE_TENTATIVES</code> , <code>EDS_RELANCE_ATTENTE_S</code>), pas des constantes
Prévenir	L'échec dépose <code>logs/ALERTE.txt</code> — code de sortie, tentatives, <code>run_id</code> de l'exécution fautive, marche à suivre — que <code>eds.run --etat</code> affiche en tête, avant même de se connecter à ClickHouse : l'alerte se voit donc même quand c'est le moteur qui est tombé. Une nuit réussie l'efface : c'est un état, pas un historique, le journal gardant la trace de l'incident. Si <code>EDS_ALERTE_CMD</code> est défini dans <code>.env</code> , le résumé est en outre poussé sur le canal du site — webhook, courriel, notification. Son échec ne devient jamais celui du pipeline, même règle que le journal ClickHouse (§ 11)
Dépendance entre tâches	Sans objet ici : une seule ligne crontab, une seule commande. Le graphe interne (<code>Pipeline.executer</code>) reste dans le processus Python, pas dans <code>cron</code>

Le canal d'alerte est délibérément hors du dépôt. Il n'est pas une propriété du pipeline mais une décision d'exploitation : le CHU a déjà une supervision, et c'est elle qu'il faut atteindre. `EDS_ALERTE_CMD` reçoit le résumé sur son entrée standard, ce qui laisse le choix du moyen sans qu'aucun outil n'entre dans `requirements.txt` — et rend la chaîne démontrable hors ligne, l'alerte fichier ne dépendant d'aucun réseau.

Le mode par défaut est incrémental, et c'est un choix. `eds.run` sans option calcule `jours_disponibles()` – `jours_deja_ingeres()` : seuls les jours absents de l'entrepôt sont chargés. Un rechargement complet (`--tout`) recopierait chaque nuit 28 fichiers pour n'en traiter réellement qu'un — au prix, à ce volume, de rester instantané, mais le principe ne tient pas au-delà de quelques dizaines de milliers de lignes par jour (§ 8.1, recalcul intégral de silver et gold). L'incrémental borne le travail nocturne à ce qui a réellement changé.

Ce qui se passe si un jour manque. `jours_disponibles()` est l'union des répertoires réellement présents sous `source-filestorage/` — pas un calendrier attendu. Si le CHU ne dépose rien une nuit, ce jour n'existe simplement pas dans la liste : l'exécution suivante n'échoue pas, elle **n'a rien à faire** pour ce jour, et se termine normalement. Le jour sera repris automatiquement dès qu'il apparaîtra dans le dépôt, sans action manuelle — mais **rien n'alerte si le CHU cesse durablement de déposer**. C'est la limite qui subsiste après l'ajout du superviseur : celui-ci prévient quand une exécution échoue, pas quand elle réussit à ne rien faire. Détecter un silence prolongé suppose un calendrier de dépôt attendu, que le CHU n'a pas fourni (le dépôt est irrégulier par conception — § 11) ; la règle reste donc à établir avec lui avant d'être codée.

10. Ce qui se passe quand ça échoue

Deux familles d'erreurs, parce qu'elles n'appellent pas la même réponse.

```
class ErreurPipeline(Exception):
    """Échec métier : la relance à l'identique ne changera rien."""

    def _est_transitoire(erreur: Exception) -> bool:
        return isinstance(erreur, OperationalError) or (
            isinstance(erreur, DatabaseError) and "Connection" in str(erreur)
        )
```

Une **panne transitoire** — le moteur ClickHouse qui redémarre, une connexion coupée — a de bonnes chances de disparaître d'elle-même en quelques secondes : elle est retentée. Une **erreur métier** — `ErreurPipeline`, ou toute erreur qui n'est ni une `OperationalError` ni une `DatabaseError` évoquant une connexion — ne l'est pas : un jour absent du dépôt, un SQL invalide, une partition déjà verrouillée resteront faux à la deuxième tentative comme à la première. La retenter ne ferait que perdre du temps et bruyter le journal d'un même échec répété trois fois.

`avec_reprises` porte la première famille : **3 tentatives**, avec **temporisation exponentielle** — 2 s avant la deuxième, 4 s avant la troisième (`ATTENTE_INITIALE_S = 2`, doublée à chaque échec). Les trois étapes qui appellent du SQL transformant (`bronze`, `silver`, `gold`) et le chargement du schéma en sont enveloppées ; la lecture du lake, elle, échoue en `ErreurPipeline` sans retenter, puisqu'un fichier absent le reste.

Deux preuves réelles, tirées de `ops.executions`, plutôt qu'un cas fabriqué. La classification n'est pas seulement écrite dans le code, elle s'observe dans l'historique des exécutions déjà journalisées :

Étape	Message (tronqué)	Famille	Comportement observé
lake	ErreurPipeline: aucun fichier trouvé pour le 2099-01-01	métier	échec immédiat, aucune retentative — répété à chaque re-jeu de <code>tests.demontrer reprise</code> , qui force un jour inexistant
silver	DatabaseError : ... Function with name `quote` does not exist	définitive (ni <code>OperationalError</code> , ni « <code>Connection</code> »)	échec immédiat malgré la classe <code>DatabaseError</code> — une erreur SQL réelle rencontrée en développement, pas retentée puisqu'aucune reprise n'aurait changé une fonction inexistante

Les deux lignes ci-dessus ont été relevées sur l'historique du développement. La seconde n'y figure plus : la table vit dans le volume ClickHouse, et sa recreation l'a remise à zéro — le pipeline, lui, n'y efface jamais rien. C'est exactement l'asymétrie qui justifie les deux destinations du § 11, et `logs/pipeline.log`, simple fichier, a tout conservé.

La répartition par étape s'obtient par :

```
SELECT etape, count() AS echecs
FROM ops.executions WHERE statut = 'echec'
GROUP BY etape ORDER BY echecs DESC;
```

Elle ne porte aujourd'hui que la ligne `lake`, tous de la famille métier, et ce compte grossit à chaque relecture de `tests.demontrer reprise`, qui force un jour inexistant. Le nombre importe peu : ce qu'il montre, c'est que la famille métier est la seule à se répéter — elle décrit une situation reproductible, quand l'échec technique ne survient qu'accidentellement.

La propriété qui rend la reprise triviale : il n'y a rien à restaurer. Bronze est la source de vérité durable — chaque partition (`_jour_depot`) est réécrite en entier (`DROP PARTITION` puis rechargement), jamais modifiée en place. Silver et gold sont **recalculés intégralement** à chaque passage (`TRUNCATE` puis `INSERT` — § 3.3). Le schéma, lui, ne détruit jamais : les six fichiers de `SCHEMA` sont tous en `CREATE TABLE IF NOT EXISTS`, rejouables sans effet de bord. La conséquence directe : après un échec, **corriger la cause et relancer suffit**. Aucune sauvegarde à restaurer, aucun script de rattrapage distinct de `eds.run` lui-même.

Ce que le pipeline garantit après un échec : un entrepôt cohérent, jamais à moitié écrit. `tests.demontrer reprise` le vérifie, pas seulement l'affirme. Rejoué à l'instant :

- | | |
|-------------------------------|---|
| ① Jour de dépôt malformé | → code de sortie 1, rejeté avant écriture |
| ② Jour absent du dépôt du CHU | → code de sortie 1, échec explicite |
| ③ Traçabilité de l'échec | → présent dans <code>ops.executions</code> |
| ④ Cohérence après incident | → inchangé : { <code>bronze.sejours: 6797</code> , <code>silver.sejours: 6729</code> , <code>gold_pilotage.fact_sejour: 6729</code> } |
| ⑤ Reprise par simple relance | → code de sortie 0, entrepôt rétabli à l'identique |
| ⑥ Chargement partiel | → <code>bronze.monitoring</code> du 2026-08-27 : 321 → 0 → 321 lignes |

Le point ⑥ est le cas qui compte réellement : une partition entière de `bronze.monitoring` est supprimée pour simuler un jour à moitié chargé — un scénario où une source aurait échoué après une autre (§ 9, `jours_deja_ingeres`, qui exige que **toutes** les sources d'un jour soient présentes, pas seulement `sejours`). Une simple relance de `eds.run` retrouve les 321 lignes disparues, sans argument spécial ni intervention : le jour est redétecté comme non entièrement ingéré, et rechargé.

Deux étages de reprise, et ils ne traitent pas le même incident. Celui décrit ci-dessus est *interne* à l'exécution : `avec_reprises` retente une opération dont la panne est transitoire — 3 tentatives, 2 s puis 4 s — sans que le pipeline s'arrête. Il ne peut rien, en revanche, contre ce qui fait échouer l'exécution entière : un ClickHouse qui ne répond plus au démarrage, un disque plein. C'est là qu'intervient le second étage, *externe* : le superviseur du § 9 relance l'exécution complète, 3 fois, à 10 min d'intervalle. La classification est la même aux deux étages — on ne retente que ce qu'une relance peut résoudre — mais l'unité diffère : une requête d'un côté, la nuit entière de l'autre.

11. La journalisation

Deux destinations, parce qu'elles servent deux usages qu'un seul support ne couvre pas. `logs/pipeline.log` est un fichier — une ligne JSON par événement, lisible par `tail -f`, par un humain en urgence à 3h du matin comme par un outil de collecte de logs (`FormateurJSON`, `eds/journal.py`). Il survit même si ClickHouse est indisponible, puisqu'il n'en dépend pas. `ops.executions` est une **table** — interrogeable en SQL, jointe aux données qu'elle décrit par `_run_id`, agrégeable (compter les échecs par étape, mesurer une durée moyenne). Le fichier répond à « que s'est-il passé, dans l'ordre ? » ; la table répond à « quelle exécution a produit cette ligne, et avec quel bilan ? ». Un seul des deux ne remplirait pas l'autre rôle : un fichier ne se `JOIN`-e pas à `bronze.sejours` sur `_run_id`, et une table disparaît si le moteur qui la porte est lui-même la panne à diagnostiquer.

Structure de `ops.executions` :

Colonne	Contenu
<code>run_id</code>	identifiant de l'exécution, partagé avec <code>_run_id</code> dans <code>bronze/silver/gold</code>
<code>etape</code>	<code>schema</code> , <code>lake</code> , <code>bronze</code> , <code>referentiels</code> , <code>silver</code> , <code>gold</code> , <code>droits</code> , <code>restitution</code>
<code>jour</code>	jour de dépôt concerné — <code>NULL</code> pour les étapes non journalières
<code>statut</code>	<code>succes</code> ou <code>echec</code>
<code>lignes</code>	volume traité par l'étape
<code>duree_s</code>	durée mesurée
<code>message</code>	vide en succès ; type et texte de l'exception en échec, tronqué à 500 caractères
<code>demarre_a</code> , <code>termine_a</code>	horodatages

Une requête d'exploitation utile — les dernières exécutions, succès comme échecs, celle que `docs/RAPPORT.md` et `README.md` recommandent en premier réflexe :

```
SELECT demarre_a, run_id, etape, statut, lignes, duree_s, message
FROM ops.executions ORDER BY demarre_a DESC LIMIT 20;
```

Ce qui distingue les deux destinations est leur grain, pas leur taille. Le fichier porte une ligne par événement — démarrage, fin d'étape, chargement d'une source, absence de dépôt — dans l'ordre où ils surviennent. La table porte une ligne par étape et par exécution, avec son statut, son volume et sa durée. Le fichier raconte, la table totalise. Les deux restent cohérentes entre elles, puisqu'écrites par le même code au même instant (`Pipeline.etape`, § 10).

Aucun compte n'est donné ici, et c'est délibéré : ce sont des journaux, pas des instantanés. Chaque relance de `eds.run` ou de `tests.demontrer` y ajoute ses lignes, si bien qu'un chiffre écrit dans ce rapport serait faux le lendemain. Les deux se comptent en une commande — `wc -l logs/pipeline.log` d'un côté, `SELECT uniq-Exact(run_id) FROM ops.executions` de l'autre.

Une asymétrie mérite en revanche d'être connue : les deux n'ont pas la même durée de vie. Le pipeline n'efface jamais une ligne, mais la table vit dans le volume ClickHouse — sa destruction lors du retour en arrière de la montée de version, racontée dans les leçons du projet, a fait repartir `ops.executions` de zéro, quand le fichier n'avait rien perdu. La table est plus riche à interroger, le fichier plus durable : c'est précisément pourquoi les deux coexistent.

Trois niveaux, et une règle qui les sépare. `INFO` décrit le déroulement normal — étape terminée, source chargée, source absente un jour où elle n'est pas déposée. `WARNING` signale ce qui mérite d'être lu sans interrompre : une colonne apparue en amont et retirée à l'entrée du lake, un fichier non déclaré qui n'a pas été copié, une connexion Metabase en double. `ERROR` accompagne l'échec d'une étape.

La règle tient en une phrase : **un avertissement qui décrit une situation normale n'en est pas un.** Le calendrier de dépôt du CHU est irrégulier par conception — `patients` est un snapshot, `actes` n'est déposée qu'une fois — si bien qu'une source sans dépôt un jour donné est le cas nominal et se journalise en `INFO`. Émise en `WARNING`, elle produisait 57 avertissements par exécution complète, et **7 421 des 7 422 avertissements de l'histoire du projet** portaient ce seul motif : le seul avertissement réel jamais émis y était noyé. Un exploitant apprend en trois exécutions que les `WARNING` ne veulent rien dire, et manque celui qui compte ; une supervision branchée sur ce niveau — la façon la plus courante de la brancher — se déclencherait chaque nuit. `tests/test_warehouse.py` fige la règle pour cette source.

Ce qui n'y entre jamais : aucune donnée de santé, aucun pseudonyme, aucun mot de passe ni jeton. Les messages ne portent que des métadonnées d'exécution — nom d'étape, jour, compte de lignes, type d'exception. Ce n'est pas seulement une promesse de l'en-tête de `eds/journal.py` : `tests.verifier rgpd` le vérifie automatiquement, à chaque passage, en balayant le fichier de log et la colonne `message` d'`ops.executions` avec un motif qui repère un pseudonyme (16 caractères hexadécimaux), un IPP (`IPP` suivi de 7 chiffres) ou un NIR (15 chiffres) :

```
OK      logs/pipeline.log sans identifiant patient
OK      ops.executions sans identifiant patient
```

Le point faible, assumé plutôt que caché. `_consigner` — la fonction qui écrit dans `ops.executions` — est elle-même enveloppée dans un `try/except` :

```
except Exception:
    # Ne jamais faire échouer le pipeline à cause de son propre
    # journal : le fichier de log reste la trace de secours.
    LOG.warning("journal ClickHouse indisponible", exc_info=True)
```

Le choix est délibéré — un journal qui peut faire échouer ce qu'il journalise serait pire que l'absence de journal — mais il a un coût : si ClickHouse tombe **au moment précis** où une étape se termine, l'écriture dans `ops.executions` échoue silencieusement (avertissement seul), et `logs/pipeline.log` devient la **seule** trace de cette étape. C'est exactement pourquoi les deux destinations coexistent plutôt qu'une seule : le fichier ne dépend d'aucun composant que le pipeline lui-même pourrait avoir mis en défaut.

12. La traçabilité, de bout en bout

Le § 3.5 annonce la propriété : chaque ligne porte son fichier d'origine et l'exécution qui l'a produite, en une requête, sans jointure entre couches. Voici la démonstration, sur un indicateur réellement affiché — « Passages aux urgences (service) », la série journalière du tableau de bord « Pilotage hospitalier » — remonté jusqu'au fichier de dépôt et à l'exécution qui l'a traité. Chaque étape est **une** requête ; aucune n'en joint deux couches.

① Le tableau de bord affiche, pour le 1er août, 46 passages.

```
SELECT jour, nb_passages_urgences FROM gold_pilotage.kpi_urgences_jour
WHERE jour = '2026-08-01';
```

```
2026-08-01    46
```

② Ce chiffre est un `countIf` sur `fact_sejour` — un séjour parmi les 46, au hasard.

```
SELECT stay_id FROM gold_pilotage.fact_sejour
WHERE date_admission = '2026-08-01' AND service_code = 'URGENCES'
ORDER BY stay_id LIMIT 1;
```

```
S00000445
```

③ Ce séjour, en silver, porte déjà son fichier et son exécution — sans jointure.

```
SELECT patient_pseudo, _fichier_source, _run_id, _built_at
FROM silver.sejours WHERE stay_id = 'S00000445';
```

```
d54c39c15c6fbf94    lake/sejours/2026-08-01/sejours.csv    a444489b6fe0    2026-09-02 21:19:44
```

④ La même ligne, en bronze, avant tout nettoyage — même fichier source, plus l'horodatage d'ingestion.

```
SELECT patient_pseudo, _fichier_source, _ingested_at, _run_id
FROM bronze.sejours WHERE stay_id = 'S00000445';
```

```
d54c39c15c6fbf94    lake/sejours/2026-08-01/sejours.csv    2026-09-02 21:08:35    40ae712db09b
```

Le `_run_id` diffère entre ③ et ④ (`a444489b6fe0` contre `40ae712db09b`) — c'est attendu, pas une anomalie. Bronze est incrémental : cette partition n'a plus été touchée depuis son premier chargement, donc son `_run_id` reste celui de ce chargement. Silver, lui, est **recalculé intégralement** à chaque exécution de `eds.run` (§ 3.3) : son `_run_id` / `_built_at` désignent la dernière reconstruction complète de la table, pas la dernière fois que cette ligne précise a changé de valeur. Les deux couches partagent toujours `_fichier_source` — la chaîne de traçabilité tient sur cette colonne, pas sur l'égalité (accidentelle, et non garantie) des `_run_id`. C'est pour la même raison que ces valeurs, rejouées après un nouveau `eds.run`, ne seront plus celles imprimées ci-dessus : elles décrivent un état observé, pas une propriété stable de l'entrepôt.

⑤ Et l'exécution `40ae712db09b`, à l'étape qui a chargé précisément ce fichier, est dans `ops.executions`.

```
SELECT etape, jour, statut, lignes, duree_s
FROM ops.executions WHERE run_id = '40ae712db09b' AND etape = 'bronze' AND jour = '2026-08-01';
```

```
bronze    2026-08-01    succes    2487    0.018
```

De la carte du tableau de bord au fichier `sejours.csv` déposé le 1er août et à l'exécution qui l'a chargé en 18 millisecondes, la chaîne tient en cinq requêtes, chacune sur une seule table. C'est la réponse concrète à « d'où vient cette donnée, et quand a-t-elle été traitée ? » — le § 3.5 en énonçait la possibilité ; ce paragraphe l'exécute.

Pourquoi gold s'arrête à `_run_id` et `_built_at`, sans `_fichier_source`. La chaîne ci-dessus le montre en pratique : dès qu'une investigation atteint gold, elle continue avec le compte d'exploitation (§ 3.4), qui lit bronze et silver — c'est là, et seulement là, que vit la colonne fichier. Répéter cette colonne dans gold n'ajouterait rien qu'un doublon, sur une base dont le compte de pilotage n'a de toute façon pas le droit de lire `stay_id`.

13. Utilisation et maintenance

TROIS NIVEAUX DE VÉRIFICATION, ET ILS NE SE REMPLACENT PAS. Les tests unitaires (`pytest`) s'exercent sur les fonctions pures : ils tournent hors ligne, en une fraction de seconde, et disent si une transformation est juste prise isolément. `tests.verifier` s'exécute contre l'entrepôt vivant et confronte chaque indicateur au recalcul depuis les faits : il dit si le système est cohérent avec lui-même. `tests.demontrer` écrit dans l'entrepôt, injecte des lignes fautives puis le remet en état : il dit si les règles FONCTIONNENT, y compris celles qu'aucune donnée réelle n'exerce.

Un test unitaire n'atteindra jamais l'équation de conservation sur 87 443 lignes, ni le refus prononcé par le moteur. Une suite qui exige Docker ne tournera jamais en trois secondes dans une chaîne d'intégration. Les trois sont donc gardés.

Commandes d'exploitation courantes :

Besoin	Commande	Effet
Lancement quotidien (celui du cron)	<code>.venv/bin/python -m eds.supervision</code>	encadre <code>eds.run</code> : verrou, relance bornée, alerte (§ 9). Les options sont transmises telles quelles
Lancement manuel	<code>.venv/bin/python -m eds.run</code>	incrémental : ingère les seuls jours absents, reconstruit silver et gold, réapplique les droits
Rejeu d'un jour précis	<code>.venv/bin/python -m eds.run -- jour 2026-08-27</code>	réécrit sa partition bronze (<code>DROP PARTITION</code> puis rechargement) ; sans effet sur les autres jours
Rechargement complet	<code>.venv/bin/python -m eds.run -- tout</code>	relit les 28 jours ; nécessaire après un changement de jeu de données source, jamais après un simple incident (§ 10)
État de l'entrepôt, sans rien modifier	<code>.venv/bin/python -m eds.run -- etat</code>	alerte en cours s'il y en a une (§ 9), jours ingérés/en attente, volumes par couche, cinq dernières étapes. Le verdict « ingéré » est rendu par <code>jours_deja_ingeres</code> , celui-là même qui commande l'exécution incrémentale : un jour sans séjour — le dépôt d'évolution du 2026-08-29 n'apporte que des actes — n'est donc pas déclaré en attente pour cette seule raison
Provisionnement de la restitution	<code>.venv/bin/python -m eds.restitution</code>	(re)crée connexions, comptes, droits et tableaux de bord Metabase — idempotent, ~6 s au premier passage, ~1,2 s ensuite
État de Metabase, sans rien modifier	<code>.venv/bin/python -m eds.restitution --etat</code>	connexions et tableaux de bord déjà provisionnés
Tests unitaires, hors ligne	<code>.venv/bin/python -m pytest</code>	169 tests sur les fonctions pures — pseudonymisation, découpage SQL, résolution des référentiels, seuils — et sur le superviseur (verrou, relance, alerte), qui écrit dans un répertoire jetable. Ne demandent ni Docker ni entrepôt , et s'exécutent en moins d'une seconde
Contrôle des 428 propriétés	<code>.venv/bin/python -m tests.verifier</code>	rejoue les cinq sections de vérification (dont <code>conformite</code> , § 6)
Démonstrations rejouables	<code>.venv/bin/python -m tests.demontrer</code>	dont <code>reprise</code> (§ 10) et <code>restitution</code> (cloisonnement vu depuis Metabase)

Deux exécutions mesurées à l'instant, pour donner un ordre de grandeur réel plutôt qu'un chiffre relu ailleurs. Une exécution sans nouveau jour à ingérer (`schema` + `referentiels` + `silver` + `gold` + `droits`) prend **0,24 s** au total sur ce dépôt (0,021 + 0,008 + 0,095 + 0,092 + 0,02, run `e4cd6aeb2e18`) ; la charge initiale des 28 jours prend, elle, environ 1,5 s — c'est ce second chiffre que mesure le démarrage décrit au README. Le provisionnement Metabase suit le même contraste : ~6 s la première fois qu'il crée tout, ~1,2 s ensuite quand il ne fait que réconcilier l'existant.

Reprise sur incident — symptôme, cause, correction. Le tableau ci-dessous reprend celui du README et le complète des cas rencontrés depuis, notamment en développement (colonne « Origine ») :

Symptôme	Cause	Correction	Origine
ClickHouse ne voit pas le lake	Le répertoire lake/ a été supprimé : le montage Docker pointe sur un inode disparu	<code>docker compose restart clickhouse</code>	<code>eds.run</code> , contrôle explicite avant lecture
Connection reset by peer / toute <code>OperationalError</code>	ClickHouse redémarre	Aucune — reprise automatique, temporisation 2 s puis 4 s (§ 10)	classification <code>_est_transitoire</code>
Erreur SQL non liée à une connexion (ex. <code>Function ... does not exist</code>)	Un fichier <code>.sql</code> a été modifié avec une erreur de syntaxe ou une fonction inexistante	Pas de reprise automatique : corriger le SQL, puis relancer	observé en développement (§ 10), 3 occurrences dans <code>ops.executions</code>
Variable d'environnement manquante	<code>.env</code> absent ou incomplet	Copier <code>.env.example</code> en <code>.env</code> et renseigner la valeur manquante — <code>eds.config.exiger</code> nomme précisément la variable en cause dans le message d'erreur	<code>eds.config.exiger</code>
argument invalide	Jour mal formé en ligne de commande	Utiliser le format <code>AAAA-MM-JJ</code>	validation avant toute connexion (<code>valider_jour</code>)
aucun fichier trouvé pour le ...	Jour absent du dépôt du CHU	Vérifier <code>eds-chu-sujet/source-filestorage/</code> ; sans action, le jour sera repris de lui-même dès son dépôt (§ 9)	<code>ErreurPipeline</code> , 72 occurrences dans <code>ops.executions</code> (dont la démonstration <code>tests.demontrer reprise</code> , rejouable, § 10)
Unknown expression identifier sur une colonne	Un DDL a été modifié : <code>CREATE TABLE IF NOT EXISTS</code> ne migre pas un schéma existant	<code>DROP TABLE ...</code> sur la table concernée, puis relancer le pipeline	§ 10, propriété « rien à restaurer »
Metabase ne répond pas sur <code>/api/health</code> après ...s	Conteneur metabase non démarré, ou JVM encore en cours de démarrage (~1 min la première fois)	<code>docker compose up -d metabase</code> , puis <code>docker compose logs metabase</code> si l'attente échoue malgré tout	<code>eds.restitution</code> , mêmes codes de sortie que <code>eds.run</code> (1 / 2)
connexions / tableaux de bord Metabase absents	<code>eds.restitution</code> n'a jamais été rejoué contre cette instance	<code>.venv/bin/python -m eds.restitution</code>	<code>tests.demontrer restitution</code>
valeurs de référence absentes, section ignorée	<code>eds-chu-sujet/corrige-kpi-niveau1.json</code> n'est pas sur cette machine (fichier non versionné, § 6)	Aucune correction : la section <code>conformite</code> est volontairement ignorée plutôt que de faire échouer le reste de la suite (§ 6)	<code>tests.verifier conformite</code>

Symptôme	Cause	Correction	Origine
Ports are not available sur Metabase	Le port 3000 est déjà occupé	Libérer le port (<code>lsof -i :3000</code>) ou republier Metabase sur un autre port	<code>docker-compose.yml</code>

Ce qu'un exploitant fait, dans l'ordre, le matin où le pipeline a échoué pendant la nuit :

1. Lire `logs/cron.log` — c'est la sortie brute de la commande cron, distincte du journal structuré : elle révèle une panne système (Python absent, disque plein, conteneur arrêté) qu' `ops.executions` ne verrait pas, puisque le pipeline n'aurait alors même pas pu s'y écrire.
2. Interroger `ops.executions` pour la nuit concernée (la requête du § 11) — identifier l'étape en échec, son message, et si l'exécution a été retentée (§ 10).
3. Chercher le symptôme dans le tableau ci-dessus — la cause et la correction sont connues pour les cas déjà rencontrés.
4. Corriger la cause, jamais l'effet — pas de tentative de réparer une table à la main : ce serait contredire la propriété du § 2.
5. Relancer `eds.run` sans option : idempotent, il retrouve tout seul ce qui manque (jour absent, partition incomplète — § 10, point ⑥).
6. Vérifier avec `eds.run --etat`, puis `tests.verifier` — 459 contrôles, dont la conformité aux valeurs de référence si le fichier est présent sur cette machine.
7. Si l'incident touchait la restitution, relancer `eds.restitution` — les tableaux de bord ne se resynchronisent pas tout seuls avec un entrepôt corrigé.

Remise à zéro complète (destructif — l'entrepôt et Metabase sont reconstruits intégralement, réservé à un incident qu'aucune correction ciblée ne résout) :

```
docker compose down -v && docker compose up -d
.venv/bin/python -m eds.run --tout
.venv/bin/python -m eds.restitution # -v supprime aussi metabase-data
```

Partie 3 — L'évolution du besoin

Le CHU a déposé de nouvelles données après la première livraison. Cette partie s'ajoute aux deux précédentes **sans les remplacer** : tout ce qu'elles décrivent reste en place et continue de produire les mêmes chiffres.

14. La demande

14.1 Ce que le CHU demande, et ce qui change

Le 29 août 2026, le CHU dépose de nouvelles données et formule une consigne en une phrase : « *faites évoluer votre entrepôt — sans tout refaire, sans rien casser* ». Trois fichiers arrivent :

Fichier	Format	Contenu
actes/2026-08-29/actes.parquet	Parquet	8 112 actes techniques — <code>stay_id</code> , <code>code_ccam</code> , <code>acte_ts</code>
referentiels/2026-08-29/ccam.csv	CSV	8 codes d'actes, leur libellé et leur tarif
referentiels/2026-08-29/description_service.csv	CSV	catégorie, capacité en lits et pôle — pour 7 services sur 8

Cinq indicateurs sont attendus : activité et DMS par catégorie de service, nombre d'actes par service, répartition par type d'acte, densité d'actes par lit, montant facturé (T2A). Le sujet énonce lui-même les deux pièges à éviter, et c'est autour d'eux que s'organisent les choix qui suivent.

Aucune section précédente de ce dossier n'a été réécrite. Les chiffres des § 1 à § 13 décrivent l'entrepôt tel qu'il était à la première livraison, et restent exacts pour ce périmètre. Deux totaux, en revanche, incluent désormais les actes : bronze passe de **79 316 à 87 443 lignes** (§ 4.4), silver de 66 369 à **74 481**, et le lake de 89 à 92 fichiers. Le tableau des sources du § 2.1 marque les référentiels « 1er jour » ; ils arrivent maintenant en **deux dépôts**, ce dont traite le § 14.2.

14.2 Un référentiel n'est pas un jour de dépôt

Le premier obstacle n'était pas dans les nouvelles données mais dans le code qui les aurait ignorées. `ingerer_referentiels` chargeait la nomenclature depuis **un seul jour** — le premier :

```
compteurs = charger_referentiels(self.ch, jours[0], self.run_id)
```

Ce choix était juste tant que tous les référentiels arrivaient ensemble. Il devient faux dès qu'un second dépôt en apporte d'autres : `ccam.csv` et `description_service.csv`, déposés le 29 août, n'auraient jamais été lus, et rien ne l'aurait signalé — le pipeline aurait tourné vert sur un entrepôt amputé.

La correction ne consiste pas à prendre le dernier jour plutôt que le premier, ce qui aurait fait disparaître `services.csv` et `cim10.csv` à la place. **Un référentiel est un FICHIER, pas un jour.** Chacun est désormais résolu indépendamment, sur le dépôt le plus récent qui le fournit (`_dernier_dépot` , dans `eds/warehouse.py`) :

```
bronze.ref_services      <- referentiels/2026-08-01/services.csv
bronze.ref_cim10         <- referentiels/2026-08-01/cim10.csv
bronze.ref_ccam          <- referentiels/2026-08-29/ccam.csv
bronze.ref_description_service <- referentiels/2026-08-29/description_service.csv
```

Cette formulation règle aussi un cas que l'ancienne ne couvrait pas : un référentiel **redéposé** remplace maintenant sa version précédente, au lieu d'être ignoré au profit du premier dépôt. Et un référentiel qu'aucun dépôt ne fournit est **signalé, sa table laissée intacte** — la tronquer remplacerait une nomenclature utilisable par une table vide, sans rien apporter.

Le journal a gagné deux champs (`table` , `fichier`) à cette occasion : quatre chargements de référentiels produisaient jusque-là quatre lignes indiscernables.

14.3 « Sans retraiter l'existant » — la propriété était déjà là

Le sujet demande d'ingérer le nouveau dépôt **par le pipeline incrémental**. Aucun développement n'a été nécessaire : la propriété existait, il suffisait de la vérifier. Une fois `actes` déclarée comme source connue, un `python -m eds.run` sans option ne traite que le jour manquant :

```
copie lake      (jour=2026-08-29)
bronze chargé   (jour=2026-08-29, lignes=8112, source=actes)
référentiel chargé (jour=2026-08-01, table=bronze.ref_services)
référentiel chargé (jour=2026-08-29, table=bronze.ref_ccam)
```

Les 28 jours antérieurs ne sont ni relus ni recopiés. Silver et gold, eux, sont intégralement recalculés — c'est le choix documenté au § 3.3 (« incrémental en amont, recalcul en aval »), et il prend tout son sens ici : une couche dérivée qui se reconstruit n'a pas de migration à subir quand son schéma change.

15. Les deux pièges

15.1 Le premier piège : un référentiel incomplet

« Le référentiel de description peut être incomplet : que faites-vous d'un service non décrit ? »

Le service **NEURO n'est pas décrit** — 7 services sur 8 le sont. Ce n'est pas un détail : la Neurologie porte **1 208 séjours et 1 471 actes**, soit 18 % de l'activité technique de l'hôpital.

Trois réponses étaient possibles, deux sont mauvaises :

- **L'exclure** (un `INNER JOIN` sur le référentiel de description). C'est ce qui se produit par défaut quand on ne se pose pas la question. 18 % de l'activité disparaît de tout indicateur, et **le total par catégorie ne vaut plus le total de l'hôpital** — un écart de 1 208 séjours que rien ne signale. C'est le mode de défaillance le plus dangereux : silencieux.
- **Lui inventer des valeurs**. Une capacité moyenne, une catégorie « médecine » par analogie. Le chiffre devient faux sans cesser d'être plausible, ce qui est pire qu'une absence.
- **Le conserver, en distinguant selon le RÔLE de la colonne**. C'est le choix retenu.

Catégorie et pôle sont des axes de regroupement. Ils valent `'non renseigné'`. La Neurologie reste comptée, la somme par catégorie fait toujours 6 729 séjours, et la lacune du référentiel **se lit dans le résultat** au lieu de s'y cacher :

Catégorie	Séjours	DMS (jours)
medecine	2 652	5,71
urgences	1 423	2,15
non renseigné	1 208	7,06
pediatrie	503	3,19
chirurgie	476	4,39
reanimation	467	9,05

La capacité en lits est un dénominateur. Elle reste `NULL`, et `kpi_densite_actes_lit` **n'a pas de ligne** pour ce service. Un ratio indéfini est absent, jamais nul : publier `0` ferait passer la Neurologie pour un service sans plateau technique, ce qui est l'inverse de la vérité.

Cette absence n'est pas une perte silencieuse, parce qu'elle est **chiffable par différence** : `kpi_actes_service` totalise 8 112 actes, `kpi_densite_actes_lit` en couvre 6 641, et l'écart de **1 471** est exactement l'activité du service non décrit. `tests.verifier` en fait un contrôle, et le tableau de bord l'explique à côté du graphe plutôt que de laisser le lecteur découvrir un service manquant.

Le résultat net : **la Neurologie figure dans quatre des cinq indicateurs**, et n'est absente que de celui dont le dénominateur lui manque.

15.2 Le second piège : le service vient du séjour

« "Actes par service" : le service est porté par le séjour, pas par l'acte — récupérez-le sans relier deux tables de faits entre elles. »

Joindre `fact_acte` à `fact_sejour` ligne à ligne serait un **fan trap** : un séjour portant trois actes verrait sa durée, son mode d'admission et son indicateur de réadmission comptés trois fois. Le § 5 documentait déjà ce risque pour justifier trois faits distincts plutôt qu'un seul ; il se représente ici sous une autre forme.

La parade suit le chemin déjà emprunté par `silver.diagnostics` et `silver.monitoring` : **le service est récupié dès silver**, depuis le séjour porteur lu dans `bronze.sejours`. `fact_acte` le lit tel quel, et aucune jointure entre faits n'existe nulle part dans la construction.

Les indicateurs qui croisent réellement les deux mondes — « nombre moyen d'actes par séjour » — agrègent **chaque fait séparément**, puis joignent les deux résultats sur `service_code`, une clé de **dimension** :

```
FROM gold_pilotage.dim_service AS d
LEFT JOIN (SELECT service_code, count() AS nb_actes
           FROM gold_pilotage.fact_acte GROUP BY service_code) AS a ...
LEFT JOIN (SELECT service_code, count() AS nb_sejours
           FROM gold_pilotage.fact_sejour GROUP BY service_code) AS sj ...
```

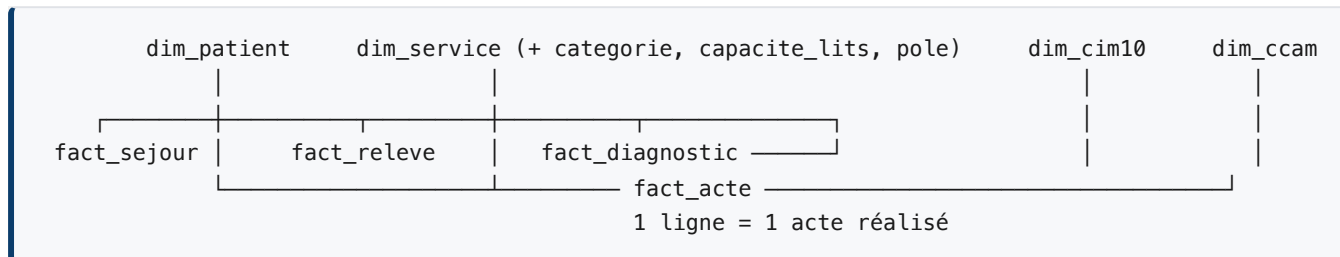
Une jointure entre deux agrégats à la même maille ne multiplie aucune ligne. C'est la jointure **ligne à ligne** qui est interdite, pas la comparaison — et `tests.verifier` le prouve plutôt que de l'affirmer : la somme des `nb_sejours` de `kpi_actes_service` vaut **6 729**, le total réel des séjours. Si le service avait été récupéré par un croisement fait-à-fait, ce nombre serait gonflé du nombre d'actes par séjour, et le contrôle échouerait.

L'indicateur lui-même est exposé en **deux lectures**, parce que « actes par séjour » est ambigu : rapporté à *tous* les séjours du service (1,21 en moyenne) ou aux seuls séjours ayant reçu un acte (1,59). Un service où un séjour sur dix reçoit dix actes et un service où chaque séjour en reçoit un donnent la même première mesure et deux secondes très différentes.

16. Le modèle après l'évolution

16.1 Un quatrième fait, une dimension de plus

Le modèle gagne un quatrième fait et une dimension, et en enrichit une autre.



fact_acte ne porte pas **patient_pseudo**, contrairement aux trois autres faits. Ce n'est pas un oubli. Les trois autres le portent parce qu'un usage nommé l'exige : cohortes de patients par pathologie, réadmission, origine géographique. **Aucun des cinq indicateurs demandés ne dénombre de patients** — ils comptent des actes, des séjours, des lits et des euros. Ajouter un pseudonyme « au cas où » contredirait exactement la minimisation défendue au § 8.2, où `region_code` n'a été conservé que parce qu'une vue d'activité lui donnait un usage. Le séjour reste joignable par `stay_id` si le besoin apparaît : ce sera alors une décision, pas un droit déjà distribué.

Le compte de pilotage, lui, n'obtient même pas `stay_id` sur ce fait — il relierait deux actes entre eux, et le pilotage analyse des volumes.

Le tarif vit dans `dim_ccam`, jamais sur le fait. C'est une donnée de facturation : elle change dans le temps sans que les actes déjà réalisés changent. Figée sur chaque ligne de fait, une révision tarifaire obligerait à réécrire l'historique.

16.2 La qualité des actes, démontrée sur des règles qui ne servent pas

`silver.actes` applique aux actes les règles déjà en vigueur pour les diagnostics et les relevés, avec trois motifs de quarantaine : `sejour_inconnu`, `sejour_ecarte`, `acte_hors_sejour`. Le contrôle physiologique n'a pas d'objet ici ; l'ordre et l'exclusivité mutuelle des trois autres sont ceux du § 4.3.

Aucun n'attrape la moindre ligne sur ce dépôt : 8 112 actes, aucun séjour orphelin, aucun code hors nomenclature, aucun acte hors de la fenêtre de son séjour. Une règle qu'aucune donnée n'exerce n'étant pas une preuve, six actes fautifs sont injectés dans `tests.demontrer qualite` — deux hors fenêtre, un orphelin, un sans patient identifié, un sur un séjour temporellement incohérent, un à code inconnu — puis l'entrepôt est remis en état.

Les 82 actes portés par les 68 séjours à incohérence temporelle sont **conservés**, avec `sejour_coherent = 0` : la décision de l'intervenant rapportée au § 4.3 vaut pour les actes comme pour les relevés. Un acte réalisé reste un acte réalisé, quelle que soit la qualité de saisie des deux dates qui encadrent le séjour.

17. Les nouveaux indicateurs

17.1 Ce qu'ils portent, et ce qu'ils prouvent

#	Table	Ce qu'elle porte	Résultat
①	kpi_activite_categorie	séjours et DMS par catégorie	6 catégories, 6 729 séjours au total
②	kpi_actes_service	actes et intensité par service	8 services, 8 112 actes, 1,21 par séjour
③	kpi_actes_type	répartition par code CCAM	8 codes, parts sommant à 100 %
④	kpi_densite_actes_lit	actes rapportés aux lits	7 lignes — URGENCES 86,55 actes/lit
⑤	kpi_facturation_service	montant T2A par service	8 services, 2 199 450 €

Activité technique et facturation

🔍 📄 ⌚ 📌 ⓘ ⋮

Activité technique et facturation

Ce tableau de bord répond aux cinq indicateurs du sujet d'ÉVOLUTION — activité et DMS par catégorie de service, actes par service, actes par type, densité d'actes par lit, montant facturé (T2A). Il s'ajoute au tableau de bord « Pilotage hospitalier », qu'il ne remplace pas : les indicateurs de la première livraison y restent inchangés.

Nombre d'actes techniques

8,112

Montant facturé (T2A)

2,199,450

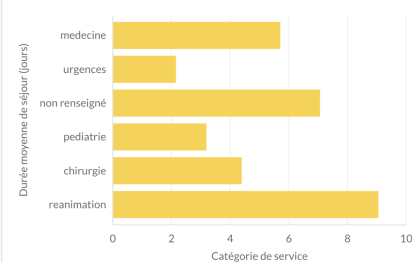
Actes par séjour

1.21

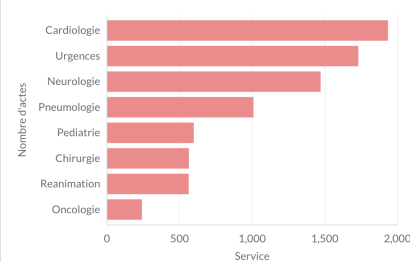
Services décrits par le référentiel

7 / 8

Activité et DMS par catégorie de service



Nombre d'actes par service

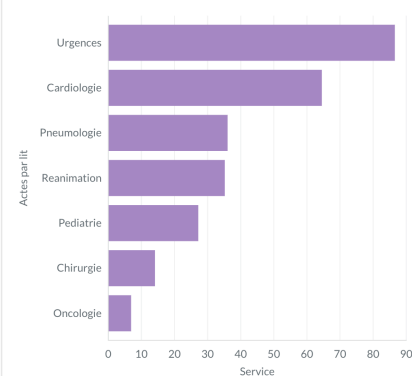


Répartition des actes par type

acte	nb_actes	part_pct	nb_sejours_concernes	tarif_euros
Radiographie du thorax	1,043	12.86	978	25
Consultation de suivi	1,039	12.81	983	25
Coronarographie	1,030	12.7	969	450
Pose de catheter central	1,025	12.64	972	120
Coloscopie totale	1,015	12.51	949	260
Ventilation mecanique assistee	1,000	12.33	946	220
IRM cerebrale	982	12.11	925	300

8 rows

Densité d'actes par lit



Pourquoi la Neurologie manque ici

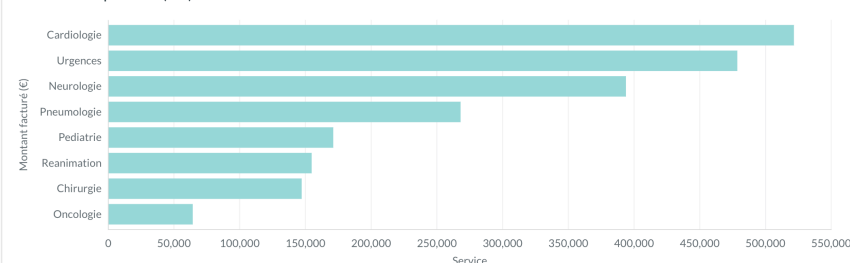
Le référentiel de description fourni par le CHU couvre 7 des 8 services : la Neurologie n'y figure pas, alors qu'elle réalise 1 471 actes, soit 18 % de l'activité technique.

Deux réponses différentes, parce que les colonnes n'ont pas le même rôle :

- **Catégorie et pôle** sont des axes de regroupement. Ils valent « non renseigné », et la Neurologie reste comptée partout — le total par catégorie fait toujours 6 729 séjours, celui de l'hôpital entier.
- **La capacité en lits est un dénominateur**. Elle est inconnue : ce graphe n'a donc pas de ligne pour la Neurologie. Afficher zéro la ferait passer pour un service sans plateau technique, ce qui serait faux.

L'écart se lit : 8 112 actes au total, 6 641 couverts ici. La différence, 1 471, est exactement l'activité du service non décrit — et non une perte.

Montant facturé par service (T2A)



Le troisième tableau de bord, ajouté sans toucher aux deux premiers. Il rend visible d'un coup d'œil ce que le § 15.1 explique : la Neurologie figure dans « Nombre d'actes par service », dans « Activité et DMS par catégorie » — sous l'étiquette « non renseigné » — et dans « Montant facturé », mais pas

dans « Densité d'actes par lit », faute de dénominateur. L'encadré placé à côté de ce graphe donne au lecteur du tableau de bord la même explication que ce rapport, sans qu'il ait à l'ouvrir.

Deux propriétés valent d'être signalées.

Les tarifs manquants sont comptés, pas absorbés. Un acte dont le code est absent de la nomenclature n'a pas de tarif : il est exclu du montant et compté dans `nb_actes_sans_tarif`. Sans cette colonne, un total sous-évalué serait indiscernable d'une activité plus faible. Sur ce dépôt, tous les codes se résolvent — la colonne vaut 0 partout, et c'est démontré par injection plutôt que constaté.

Chaque table est confrontée à son recalcul depuis les faits. Une table d'indicateur est une copie dérivée : elle peut diverger. Les cinq nouvelles rejoignent donc le harnais du § 6, et dix-sept contrôles supplémentaires vérifient ce qui leur est propre — conservation des totaux, absence de ligne pour un dénominateur inconnu, écart chiffré avec les services non décrits.

17.2 Ce qu'il a fallu accepter

Ajouter une colonne à une table gold a demandé un geste manuel, une fois. Le pipeline ne supprime aucune table : ses deux seuls `DROP` portent sur une partition de bronze, pour rendre l'ingestion d'un jour rejouable. Mais `30_gold.sql` n'exécute que des `CREATE TABLE IF NOT EXISTS`, qui ne font rien sur une table déjà présente : pour que `dim_service` gagne `categorie`, `capacite_lits` et `pole`, il a fallu la supprimer une fois à la main, le temps qu'elle se recrée au schéma voulu.

Le geste est sans risque, et c'est ce qui l'a rendu acceptable : **gold est intégralement dérivé de silver et se reconstruit en une exécution.** Rien ne s'y perd, puisque rien n'y est saisi. Il reste néanmoins manuel, et une évolution ultérieure du schéma gold le redemandera. L'alternative — un `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` par colonne — ajouterait une migration versionnée à maintenir pour une couche qui n'a précisément aucune migration à subir. Le geste ponctuel a donc été préféré, mais il doit être connu de qui reprendra le projet.

17.3 La règle posée plutôt que le cas traité

Une source de plus, donc une règle générale plutôt qu'un cas particulier. `actes` arrive en Parquet, un format binaire, et sans transformation à appliquer : la protection écrite pour `patients` — une liste blanche de colonnes — ne la couvrait pas. Deux réponses étaient possibles : décrire les colonnes d'`actes` seule, ou faire de la déclaration la règle commune à toutes les sources. La seconde a été retenue, et c'est le § 4.5. Décrire `actes` seule aurait laissé la question entière pour la source suivante, et une protection qui dépend de la vigilance de celui qui ajoute la source n'en est pas une. Même raisonnement que pour la résolution des référentiels par fichier, traitée plus haut : l'évolution ne demandait qu'un cas, elle a été l'occasion de poser la règle.

Le même raisonnement a tranché le niveau des avertissements. `charger_bronze_jour` journalise `source absente` pour chaque jour sans dépôt d'une source donnée. `actes` n'étant déposée qu'une fois, elle en ajoutait 28 par exécution complète, portant le total de 29 à 57. Là encore, deux réponses étaient possibles : taire le cas d'`actes`, ou reconnaître que le motif n'était pas un avertissement du tout. Le calendrier de dépôt du CHU est irrégulier par conception ; une source absente un jour où elle n'est pas déposée décrit le normal. Elle est donc journalisée en `INFO`, et `WARNING` redevient ce qu'il doit être — le niveau de ce qui mérite d'être lu. Le § 11 énonce la règle pour les trois niveaux.

Partie 4 — Le déploiement cloud

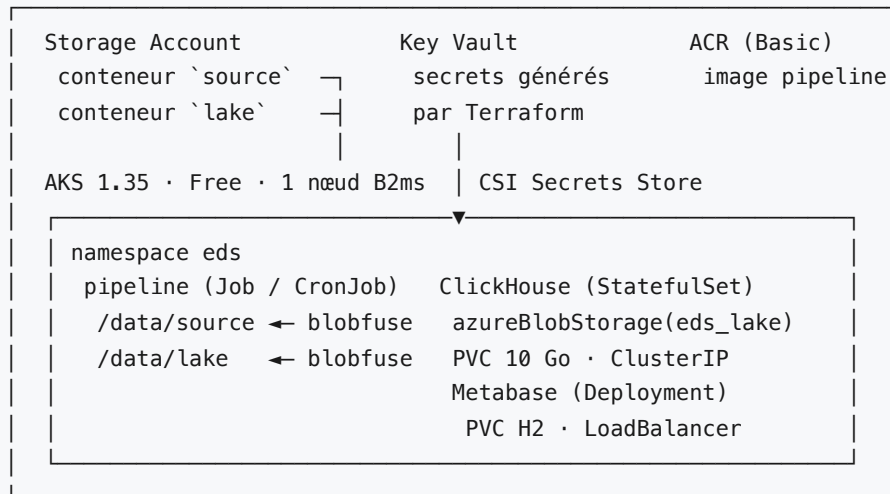
18. Ce qui est déployé, et pourquoi là

18.1 Le même entrepôt, ailleurs

LA RÈGLE QUI COMMANDE TOUT LE RESTE : AUCUNE BIFURCATION DANS LE CODE. Le pipeline qui tourne sur Azure est celui des trois parties précédentes, à l'octet près — même image des sources, même lake, mêmes fichiers SQL, mêmes droits. Ce qui change entre le poste et le cluster tient en cinq variables d'environnement, et l'absence de chacune conserve le comportement local :

Variable	Sur le poste (absente)	Dans le cluster	Qui la lit
EDS_SOURCE	eds-chu-sujet/source-filestorage/	/data/source , conteneur Blob monté	eds.config , à l'import
EDS_LAKE	lake/	/data/lake , conteneur Blob monté	idem
EDS_LAKE_LECTEUR	fichier — ClickHouse lit file()	blob — ClickHouse lit azure-BlobStorage()	eds.warehouse.-source_lake
CH_HOST	localhost (port publié par Docker)	clickhouse (nom du Service)	eds.warehouse.-client
MB_URL	http://localhost:3000	http://metabase:3000	eds.restitution

Une bifurcation aurait été plus rapide à écrire — un `if cloud:` dans le chargeur bronze — et aurait produit deux pipelines qui divergent sans que rien ne le signale. Ici, `source_lake` est la **seule** fonction qui sache qu'Azure existe : elle reçoit un chemin relatif, un format et une structure, et rend l'expression de table que le moteur doit lire. Les cinq chargeurs et les référentiels l'appellent ; un test unitaire vérifie qu'en mode `blob` aucun d'eux n'émet plus de `file()`. La colonne de provenance `_source_path` garde la même forme dans les deux modes (`lake/<chemin>`) : une ligne chargée en local et la même ligne chargée en cloud portent la même traçabilité, c'est également testé.



Le socle est décrit en Terraform (`infra/terraform/` , onze fichiers, un état local), les composants en manifestes Kubernetes (`infra/k8s/base/`), l'image dans `infra/Dockerfile` . Un script, `ops/cloud.sh` , enchaîne ce qu'aucun des trois outils ne fait seul : construire l'image dans le registre, envoyer le dépôt source, rendre les identifiants du déploiement dans les manifestes, poser, attendre. Quatre verbes — `deployer` , `charger` , `restituer` , `destruire` — et l'équivalent cloud de « quatre commandes » du README ; trois de plus pour l'exploitation (`etat` , `image` , `rendre`).

18.2 AKS plutôt que Container Apps

Trois charges de travail — un moteur avec disque, une application web, un lot nocturne — n'ont pas besoin d'un cluster Kubernetes. Azure Container Apps les aurait portées sans nœud à gérer, pour moins cher, et la question mérite d'être posée franchement plutôt que tranchée par habitude.

AKS a été retenu pour ce que la démonstration doit **montrer**, pas pour ce dont l'entrepôt a besoin. Un StatefulSet avec son disque persistant, un CronJob avec sa politique de concurrence, un pilote CSI qui projette des secrets, un autre qui monte un conteneur Blob comme un répertoire : ce sont des objets que le sujet demande de savoir manipuler, et qui n'ont pas d'équivalent lisible ailleurs. Le prix de ce choix est tenu au plus bas que l'abonnement permette : tier Free (le plan de contrôle n'est pas facturé), un seul nœud `Standard_B2ms` — 2 vCPU et 8 Go, dans le quota étudiant de 4 vCPU sur la famille B —, aucune zone de disponibilité, pas de Log Analytics. Ce qui manque à ce cluster pour être un cluster de production est listé au § 21, et c'est volontaire.

18.3 Blob plutôt qu'un volume partagé

Sur le poste, ClickHouse et le pipeline partagent un répertoire : le premier lit ce que le second écrit, par un montage Docker. Dans un cluster, le réflexe serait un volume `ReadWriteMany` monté dans les deux pods — Azure Files, en l'occurrence. Il aurait fonctionné sans toucher au code, et il a été écarté pour deux raisons.

La première est de fond : un entrepôt cloud lit son lake par l'API de l'objet, pas à travers un système de fichiers réseau émulé. ClickHouse le sait faire nativement — `azureBlobStorage()` lit un blob comme `file()` lit un fichier, avec les mêmes formats et la même colonne `_path` . La seconde est de prudence : un montage SMB partagé entre deux pods, sur un nœud unique, ajoute une couche dont aucune des deux parties n'a besoin.

Le pipeline, lui, reste sur un système de fichiers : `eds.lake` lit et écrit des chemins, avec `csv`, `json` et DuckDB, et le sujet impose que les identités ne touchent jamais un disque intermédiaire. Réécrire cette transformation en flux vers l'API Blob aurait été possible, et aurait remplacé un module éprouvé par un module neuf, pour rien. Les deux conteneurs sont donc montés dans le seul pod du pipeline, par blobfuse : la pseudonymisation lit `/data/source` et écrit `/data/lake`, en flux, comme sur le poste ; le blob est écrit à la fermeture du fichier, et ClickHouse le lit par l'API l'instant d'après. La sonde d'accès au lake — un témoin écrit par Python puis lu par le moteur avant toute ingestion, § 10 — prouve à chaque exécution que les deux chemins convergent bien sur le même conteneur.

19. La sécurité, dans l'ordre où elle est prononcée

LA RÉGION D'ABORD. Un entrepôt de santé français se déploie sur un hébergeur certifié HDS ; France Central l'est, les autres régions Azure européennes aussi mais elles n'apportent rien de plus ici. La donnée qui entre dans le conteneur `source` porte des identités en clair — c'est le dépôt du CHU tel qu'il est déposé, et la Partie 1 explique pourquoi on ne le transforme pas avant de le lire. Le jeu est synthétique ; le principe est appliqué comme s'il ne l'était pas.

LES SECRETS NE SONT LUS PAR PERSONNE. Terraform génère les quatre mots de passe ClickHouse, le sel de pseudonymisation, les trois comptes Metabase, et les dépose dans un Key Vault. L'addon Secrets Store CSI de l'AKS, avec sa propre identité managée et le seul rôle « Key Vault Secrets User », les projette dans chaque pod et les recopie dans un Secret Kubernetes que les conteneurs lisent en variables d'environnement. Aucun manifeste ne porte une valeur ; aucun humain n'a eu à en choisir une ; le `.env` du poste n'a pas d'équivalent versionné. Le sel, en particulier, n'existe qu'à deux endroits : le coffre et la mémoire du pod qui pseudonymise.

LA CLÉ DU STOCKAGE N'EXISTE QU'À UN ENDROIT. blobfuse monte les deux conteneurs avec l'identité managée du nœud, à qui Terraform donne le rôle « Storage Blob Data Contributor » sur le compte : pas de clé pour ce chemin. L'envoi du dépôt source par l'opérateur passe par son identité à lui (`--auth-mode login`) : pas de clé non plus. Reste ClickHouse, dont le connecteur Azure s'authentifie par chaîne de connexion ; elle est rendue par Terraform dans un fichier XML — la named collection `eds_lake` — stocké dans le Key Vault comme un secret parmi les autres, et monté dans le seul pod du moteur, sous `config.d`. Une requête SQL nomme la collection, jamais la clé : `azureBlobStorage(eds_lake, blob_path='patients/2026-08-26/patients.csv', ...)`. C'est la même discipline que le § 3.4 : le secret est là où le moteur le lit, pas dans ce que l'applicatif envoie.

CLICKHOUSE N'EST PAS SUR INTERNET. Son Service est de type ClusterIP : seuls Metabase et le pipeline, dans le même espace de noms, le joignent. Metabase est le seul composant exposé, derrière un équilibreur de charge dont `loadBalancerSourceRanges` n'admet que l'adresse de l'opérateur — un paramètre Terraform, validé comme CIDR. Ni TLS ni nom de domaine devant : une démonstration d'une heure depuis une seule adresse ne les justifie pas, et le § 21 les inscrit en limite.

LE CLOISONNEMENT N'A PAS BOUGÉ. Il est prononcé par `sql/50_droits.sql`, joué à chaque exécution, et le réseau n'y ajoute rien. La démonstration du § 3.4 se rejoue depuis l'intérieur du cluster, avec le mot de passe lu dans le volume projeté plutôt qu'affiché :

```
kubectl -n eds exec clickhouse-0 -- sh -c 'clickhouse-client --user eds_pilotage \
--password "$(cat /mnt/secrets/ch-pilotage-password)" \
-q "SELECT count() FROM gold_recherche.coh_prevalence"'
# → Code: 497. DB::Exception: eds_pilotage: Not enough privileges. To execute
# this query, it's necessary to have the grant SELECT for at least one column
# on gold_recherche.coh_prevalence. (ACCESS_DENIED)
```

Le compte de pilotage lit `gold_pilotage.kpi_dms_service` ; il ne voit pas `gold_recherche`, ni dans le cluster ni ailleurs. Metabase, branché sur ces deux comptes, en hérite exactement comme en local (§ 7.4). Les trois tableaux de bord, provisionnés dans le cluster par le Job `eds-restituer` (31 questions vérifiées par l'API), sont ceux de la Partie 1, servis depuis l'adresse publique de l'équilibreur :

Pilotage hospitalier

Ce tableau de bord répond aux quatre indicateurs nommés du §4 du sujet — durée moyenne de séjour, passages aux urgences, réadmission à 30 jours, alertes de monitoring — et aux vues complémentaires d'occupation, de case-mix et d'origine géographique des séjours.

Source : couche gold_pilotage de l'entrepôt — des agrégats déjà construits, jamais les faits

Dernière construction de l'entrepôt (co...

sept. 4, 2026, 2:06 PM

Nombre de séjours

6,729

DMS globale (toutes duré...

5.15

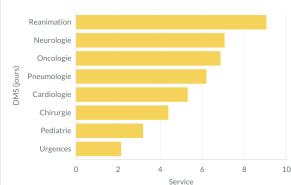
Taux de réadmission à 30 ...

11.59 %

Taux de relevés en alerte

8.1 %

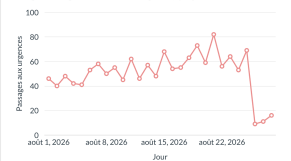
DMS par service



Dispersion des durées de séjour par service

service	moyenne_jours	medians_jours	p90_jours	max_jour
Reanimation	9.05	8.21	18.03	18.1
Neurologie	7.06	5.83	12.5	12.1
Oncologie	6.87	5.67	12.32	12.1
Pneumologie	6.2	5.71	10.33	10.1
Cardiologie	5.31	4.71	9.42	9.1
Chirurgie	4.39	3.67	8.21	8.1
Pédiatrie	3.19	2.58	6.26	6.1

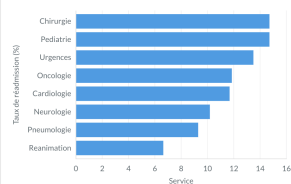
Passages aux urgences (service des urgences), par jour



Admissions en urgence (mode d'admission), tous services, par jour



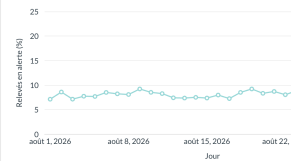
Réadmission à 30 jours par service (taux brut)



Réadmission à 30 jours par service — numérateur et dénominateur

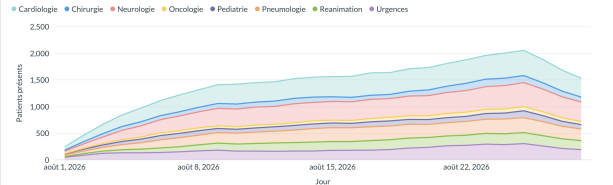
service	nb_readmis_30j_brut	nb_sejours	taux_brut_pct
Chirurgie	70	476	14.71
Pédiatrie	74	503	14.71
Urgences	192	1,423	13.49
Oncologie	25	211	11.85
Cardiologie	187	1,601	11.68
Neurologie	123	1,208	10.18
Pneumologie	78	840	9.29

Relevés en alerte par jour



Seuls deux services sont équipés d'un monitoring continu : Cardiologie et Réanimation. Le reste de l'hôpital n'apparaît dans aucun relevé — l'absence de courbe pour un autre service n'est donc pas une panne de collecte, mais l'absence de l'équipement lui-même. Le taux affiché ci-contre ne porte que sur les deux services suivis.

Occupation par jour et par service



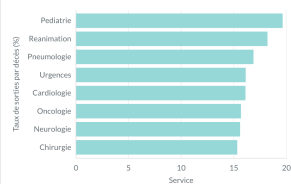
Case-mix par service

service	pathologie	nb_sejours	part_e
Cardiologie	Diabète sucre de type 2	759	
Cardiologie	Insuffisance cardiaque	547	33
Cardiologie	Infarctus aigu du myocarde	309	15
Chirurgie	Appendicite aigüe	480	
Neurologie	Episode depressif	751	61
Neurologie	Infarctus cerebral (AVC ishemique)	462	2
Neurologie	Amyotrophie spinale	5	0

Origine géographique des séjours, par service

service	region_code	nb_sejours	nb_patients	part_pct
Cardiologie	75	228	213	14.24
Cardiologie	95	227	214	14.18
Cardiologie	93	209	202	13.05
Cardiologie	77	206	197	12.87
Cardiologie	78	190	187	11.87
Cardiologie	91	190	184	11.87
Cardiologie	92	183	172	11.43

Mortalité par service



Réserve méthodologique. Dans ce jeu de données synthétique, le mode de sortie de chaque séjour est tiré uniformément au hasard, indépendamment du service et de la pathologie. Le taux affiché ci-contre — y compris un chiffre élevé en pédiatrie — n'a donc aucune portée clinique : il ne mesure ni la qualité des soins ni la gravité des patients pris en charge. Il est publié tel quel, avec cette réserve, par fidélité à ce que contient réellement l'entrepôt.

Édité il y a 11 minutes par vous

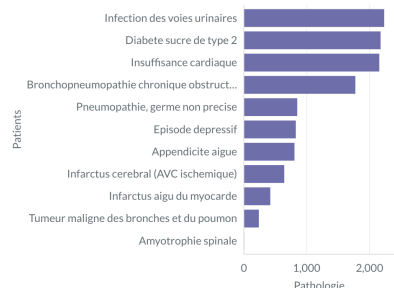
Nombre de pathologies diffusables

11

Effectif total décrit

6,501

Prévalence par pathologie



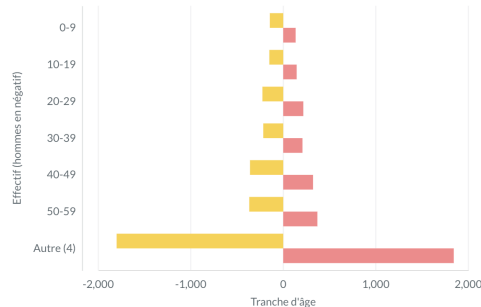
Prévalence par pathologie – référence et motif principal

pathologie	nb_patients	nb_patients_principal
Infection des voies urinaires	2,234	8
Diabete sucre de type 2	2,177	8
Insuffisance cardiaque	2,156	7
Bronchopneumopathie chronique obstructive	1,775	2
Pneumopathie, germe non precise	850	8
Episode depressif	827	8
Appendicite aigue	806	8
Infarctus cerebral (AVC ishemique)	643	6

11 lignes

Description de cohorte – pyramide des âges

● hommes ● femmes



Description de cohorte — détail par pathologie, âge et...

pathologie	tranche_age	sexe	nb_patients
Appendicite aigue	0-9	F	67
Appendicite aigue	0-9	M	84
Appendicite aigue	10-19	F	127
Appendicite aigue	10-19	M	129
Appendicite aigue	20-29	F	64
Appendicite aigue	20-29	M	123
Appendicite aigue	30-39	F	68
Appendicite aigue	30-39	M	119
Appendicite aigue	40-49	F	7
Nonappendicite aigue	40-49	M	18

89 lignes

Seuil de petit effectif. Aucune cohorte de moins de 5 patients n'est diffusée dans cette base : le filtre $k \geq 5$ est appliqué à l'ÉCRITURE de `coh_prevalence` et de `coh_description`, il n'y a donc rien à masquer à la lecture — une requête ne peut pas contourner une ligne qui n'existe pas. Le seuil agit à deux granularités différentes, visibles séparément ici :

- **Par pathologie entière.** La mucoviscidose (E84) et la trisomie 21 (Q90) ont un effectif total sous le seuil : elles n'apparaissent dans AUCUNE des deux tables, ni en prévalence ni en description.
- **Par cellule.** L'amytrophie spinale (G12) franchit le seuil globalement — elle figure donc dans « Prévalence par pathologie » — mais chacune de ses sept cellules pathologie $\times$ tranche d'âge $\times$ sexe reste individuellement sous 5 patients : elle est donc totalement absente de la description de cohorte, sans qu'une colonne ligne isolée ne la laisse deviner.

An total, **treize cellules** sont retenues par le seuil : les sept de l'amyotrophie spinale, plus les trois de chacune des deux pathologies ci-dessus – dont l'effectif global, déjà sous le seuil, ne pouvait de toute façon fournir aucune cellule diffusable.

Activité technique et facturation

Ce tableau de bord répond aux cinq indicateurs du sujet d'ÉVOLUTION – activité et DMS par catégorie de service, actes par service, actes par type, densité d'actes par lit, montant facturé (T2A). Il s'ajoute au tableau de bord « Pilotage hospitalier », qu'il ne remplace pas : les indicateurs de la première livraison y restent inchangés.

Nombre d'actes techniques

8,112

Montant facturé (T2A)

2,199,450

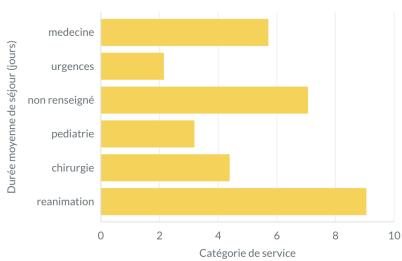
Actes par séjour

1.21

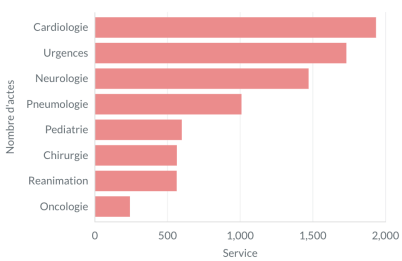
Services décrits par le référentiel

7 / 8

Activité et DMS par catégorie de service



Nombre d'actes par service

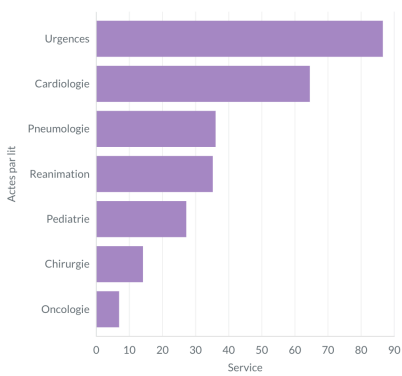


Répartition des actes par type

acte	nb_actes	part_pct	nb_sejours_concernes	tarif_euros
Radiographie du thorax	1,043	12.86	978	25
Consultation de suivi	1,039	12.81	983	25
Coronarographie	1,030	12.7	969	450
Pose de catheter central	1,025	12.64	972	120
Coloscopie totale	1,015	12.51	949	260
Ventilation mecanique assistee	1,000	12.33	946	220
IRM cerebrale	982	12.11	925	300

8 lignes

Densité d'actes par lit



Pourquoi la Neurologie manque ici

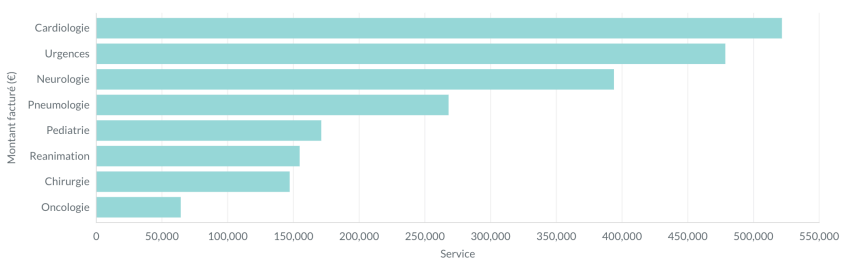
Le référentiel de description fourni par le CHU couvre 7 des 8 services : la Neurologie n'y figure pas, alors qu'elle réalise 1 471 actes, soit 18 % de l'activité technique.

Deux réponses différentes, parce que les colonnes n'ont pas le même rôle :

- **Catégorie et pôle** sont des axes de regroupement. Ils valent « non renseigné », et la Neurologie reste comptée partout – le total par catégorie fait toujours 6 729 séjours, celui de l'hôpital entier.
- **La capacité en lits est un dénominateur**. Elle est inconnue : ce graphe n'a donc **pas de ligne** pour la Neurologie. Afficher zéro la ferait passer pour un service sans plateau technique, ce qui serait faux.

L'écart se lit : 8 112 actes au total, 6 641 couverts ici. La différence, 1 471, est exactement l'activité du service non décrit – et non une perte.

Montant facturé par service (T2A)



20. La nuit, sans superviseur

Sur le poste, `cron` déclenche et ne fait rien d'autre ; `eds.supervision` met en face un verrou, une relance bornée et une alerte (§ 9, § 10). Dans le cluster, le CronJob `eds-nuit` lance `eds.run` directement, à 03h10 heure de Paris, et le module de supervision n'est pas appelé. Ce n'est pas une régression, c'est une redistribution : ce que le module apportait à `cron` est ce qu'un orchestrateur fournit nativement.

Le superviseur faisait	Le CronJob fait	Différence
Verrou par PID, repris s'il est périmé	<code>concurrencyPolicy: Forbid</code> : un Job encore actif empêche le suivant de démarrer	Le verrou vivait dans un fichier ; ici c'est l'API qui connaît l'état, il n'y a pas de verrou orphelin possible
3 relances espacées de 10 min, sur échec inattendu seulement	<code>backoffLimit: 3</code> , temporisation exponentielle	Le CronJob relance tout échec, y compris métier (code 1). Sur une erreur SQL, il relancera trois fois pour rien : c'est la limite acceptée en échange de la simplicité
<code>logs/ALERTE.txt</code> + commande de site	L'état du Job (<code>kubectl -n eds get jobs</code>) et les trois derniers Jobs conservés, réussis comme échoués	Il n'y a pas de fichier d'alerte parce qu'il n'y a pas de disque durable où le déposer : un pod est éphémère, l'état vit dans l'orchestrateur

Le journal structuré (§ 11) est inchangé : `eds.journal` écrit sur la sortie standard en plus du fichier, et `kubectl logs` le lit tel quel. La table `ops.executions` (§ 12) reste la trace durable, dans ClickHouse, sur le disque persistant du StatefulSet.

21. Limites et coût

Ce déploiement est une démonstration, et chacune des lignes ci-dessous est ce qu'il faudrait ajouter pour qu'il cesse de l'être. Aucune n'est cachée dans une configuration « à faire plus tard » : elles sont nommées.

Limite	Conséquence	Ce que serait la version durable
Base applicative Metabase en H2, sur un disque	Un pod redémarré retrouve sa configuration, mais H2 n'est pas prévu pour la concurrence ni les sauvegardes	Azure Database for PostgreSQL, et <code>MB_DB_TYPE=postgres</code>
Un nœud, une réplique de chaque composant	Une panne de nœud arrête tout le temps du redémarrage	Deux nœuds au moins, ClickHouse répliqué par Keeper
Pas de TLS devant Metabase	Le mot de passe transite en clair jusqu'à l'équilibreur, depuis une seule adresse	Un ingress avec certificat, un nom de domaine
Pas d'observabilité centralisée	Les journaux vivent le temps du pod	Log Analytics ou équivalent, et une alerte sur l'échec du CronJob
État Terraform local	Un seul poste peut faire évoluer l'infrastructure	Un backend Blob avec verrou
Relance sur erreur métier	Trois exécutions inutiles avant de constater une erreur SQL	Une distinction des codes de sortie dans le pod, ou un opérateur de workflow
Le dépôt source en cloud	Des identités en clair dans un conteneur Blob, hors du CHU	Un dépôt côté CHU, et une pseudonymisation qui s'exécute avant la sortie de son réseau

Coût. Cluster allumé, France Central, tarifs relevés le 4 septembre 2026 : le nœud B2ms est le poste principal, autour de 2 €/jour ; l'équilibreur de charge Standard et son adresse publique, 0,6 € ; le registre Basic, 0,15 € ; les trois disques managés et le stockage Blob, quelques centimes. Soit environ **2,8 €/jour** et moins d'un euro pour une démonstration de trois heures. `ops/cloud.sh detruire` supprime le groupe de ressources entier, adresse publique comprise, et purge le coffre — il ne reste rien à facturer. Sur un crédit étudiant de 85 €, c'est l'équivalent d'un mois continu, ou d'une centaine de démonstrations.

La démonstration qui a produit les captures du § 19 a tenu le cluster allumé un peu moins de trois heures, le 4 septembre 2026, chargement complet et provisionnement compris ; après `detruire`, `az group list` ne montrait plus le groupe, `az keyvault list-deleted` plus le coffre. La consommation remonte avec plusieurs heures de retard dans l'API de facturation : le relevé du jour même affichait encore zéro, la valeur réelle se lit le lendemain dans le portail, poste « Azure for Students ».

Validation des chiffres

Un chiffre publié sans justification n'est pas un résultat, c'est une affirmation. Ce chapitre expose comment chacun de ceux qui précèdent est établi, et comment le rejouer.

Tout ce qui suit est recalculé, jamais recopié. Les valeurs ci-dessous sortent de l'entrepôt au moment de la rédaction ; `python -m tests.verifier` les rejoue à la demande et échoue si l'une d'elles a bougé. Le jeu de données n'étant pas versionné, aucun chiffre n'est écrit en dur dans un contrôle : ce qui est vérifié, c'est une **relation** entre des tables, pas une constante.

L'équation de conservation, couche par couche

Aucune ligne ne disparaît sans être comptée. Pour quatre des cinq sources, la relation `bronze = silver + quarantaine('ecarte')` se ferme exactement :

Source	Bronze	Silver	Écartés	Équation
sejours	6 797	6 729	68	✓
diagnostics	12 720	12 720	0	✓
monitoring	41 778	40 920	858	✓
actes	8 112	8 112	0	✓
patients	18 000	6 000	0	voir ci-dessous

Le cas **patients** n'est pas une anomalie, c'est un grain différent. La source est un **instantané cumulatif** : trois dépôts, chacun portant les 6 000 patients, soit 18 000 lignes bronze pour 6 000 individus. Silver n'écarte rien — il **réduit**, en retenant la version du dépôt le plus récent. L'équation ligne à ligne ne s'y applique donc pas ; ce qui est vérifié à sa place est qu'aucun patient n'est perdu et qu'aucun n'est dupliqué :

```
uniqExact(patient_pseudo) en bronze = 6 000
count() en silver                  = 6 000
```

Confondre les deux propriétés serait l'erreur à ne pas commettre : une déduplication qui perdrait 200 patients fermerait quand même une équation écrite trop vite.

926 lignes en quarantaine au total, toutes en `action = 'ecarte'`, chacune avec son motif et le fichier dont elle provient. Le détail des treize motifs — dont huit qui n'attrapent aucune ligne sur ce dépôt et sont donc démontrés par injection — est au § 4.3.

Les indicateurs, confrontés à leur recalcul depuis les faits

Une table d'indicateur est une **copie dérivée** : elle peut diverger de ce qu'elle prétend résumer. Un `TRUNCATE` oublié, un filtre modifié d'un seul côté, et elle continue de servir un chiffre que plus rien ne fonde.

Chacune des treize tables `kpi_*` est donc confrontée à son recalcul depuis les tables de faits — même clé, même valeur, même nombre de lignes. C'est le harnais décrit au § 6 ; il tourne à chaque exécution de `tests.verifier`.

Indicateur	Valeur	D'où elle sort
Séjours	6 729	<code>fact_sejour</code>
Patients	6 000	<code>dim_patient</code>
Séjours en cours	683	<code>fact_sejour</code> , <code>est_en_cours = 1</code>
DMS globale	5,15 j	moyenne sur les séjours clos
Réadmission à 30 jours (brute)	11,59 %	780 / 6 729, décès compris
Passages aux urgences	1 423	<code>service_code = 'URGENCES'</code>
Relevés en alerte	3 314 · 8,10 %	<code>fact_releve</code> , seuils du § 4.1
Pathologies diffusables	11 sur 13	<code>coh_prevalence</code> , $k \geq 5$
Cohortes de description	89	<code>coh_description</code> , $k \geq 5$

Les deux dernières lignes portent la preuve du seuil de diffusion. Le référentiel compte treize pathologies ; onze seulement atteignent la base recherche. La trisomie 21 (3 patients) et la mucoviscidose (4) en sont absentes — non filtrées à la lecture, **absentes de la table**. Personne n'a eu à le demander.

Confrontation aux valeurs de référence de l'intervenant

L'intervenant a fourni une feuille de réponses attendues. Elle est **hors dépôt** — marquée « ne pas distribuer » — et transcrite dans un fichier JSON lui aussi exclu du versionnement.

`python -m tests.verifier conformite` confronte l'entrepôt à ces valeurs, une par une : volumétries de silver, effectifs par pathologie, moyennes à $\pm 0,1$. Cette section est d'une **nature différente** des quatre autres : elle peut échouer alors que toutes les propriétés tiennent. Un dénominateur peut être cohérent avec lui-même — se recalculer à l'identique depuis le fait dont il sort, ne jamais dépasser son numérateur — sans être **le bon**.

C'est exactement ce qui s'est produit sur deux définitions. Compter les passages aux urgences au **mode d'admission** (3 327) ou au **service** (1 423) sont deux lectures également cohérentes ; seule la confrontation à la référence a tranché laquelle est attendue. Idem pour la réadmission, dont la définition de référence inclut les décès.

Sur une machine où le fichier de référence est absent — le cas d'un clone du dépôt — la section s'annonce **ignorée** et le reste de la suite s'exécute normalement. Un contrôle qui ne peut pas s'exercer le dit, il ne se tait pas.

Ce que cette validation ne couvre pas

Aucune vérification médicale. Les données sont synthétiques (§ 8.1) : la plateforme restitue fidèlement ce qu'elles contiennent, et aucune conclusion clinique n'en est tirable. Le taux d'alerte plat entre les deux services équipés — 8,1 % en Cardiologie, 8,0 % en Réanimation — est d'ailleurs le signe qu'elles sont générées.

Aucun recalcul entièrement indépendant du pipeline. Les contrôles confrontent des tables de l'entrepôt entre elles, et l'entrepôt à la feuille de référence. Ils ne relisent pas les fichiers source avec un second code écrit séparément. C'est la limite de ce dispositif : une erreur qui affecterait identiquement le fait et son indicateur ne serait détectée que par la confrontation à la référence — laquelle ne couvre pas tous les chiffres.

Le nombre de contrôles n'est pas une garantie. 428 propriétés vérifiées et 169 tests unitaires ne disent rien de ce qui n'est pas mesuré. Deux des défauts racontés au chapitre suivant sont passés à travers tous ces contrôles : ils n'ont été trouvés qu'en fabriquant une donnée qui n'existait pas, et en lisant le document produit.

Ce que ce projet nous a appris

Ces leçons ne sont pas des principes lus ailleurs. Chacune vient d'une erreur commise sur ce projet, chacune a changé quelque chose dans le code, et chacune est vérifiable dans l'historique du dépôt.

Un contrôle peut être juste et ne rien contrôler

`coalesce(c.libelle, 'inconnu')` après un `LEFT JOIN` n'a jamais rien fait. Avec `join_use_nulls = 0` — le défaut de ClickHouse — une absence de correspondance remplit une colonne `String` avec la **chaîne vide**, pas avec `NULL` : le repli ne se déclenche donc jamais. Trois enrichissements par libellé portaient ce défaut, dont deux depuis la première livraison.

Le code documentait pourtant le piège, et s'en prémunissait ailleurs : le drapeau `sejour_coherent` teste `sj.s-tay_id != ''`, jamais `IS NOT NULL`. La parade était connue, écrite, commentée — et deux lignes plus loin le même piège se refermait.

Rien ne l'a révélé pendant des semaines, parce que tous les codes du dépôt se résolvent. Ce qui l'a révélé, c'est une démonstration écrite pour prouver qu'un code CCAM **inventé** produirait le libellé « inconnu ». Il produisait une chaîne vide.

Ce que nous en retenons. Un contrôle qui ne s'exerce sur aucune donnée réelle n'est pas du zèle : c'est le seul endroit où un défaut latent devient visible avant qu'un dépôt futur ne le rende coûteux. Et un piège documenté n'est pas un piège évité — il faut le tester, pas le commenter.

Une règle qui n'attrape rien doit quand même être démontrée

Sur le dépôt livré, huit motifs de quarantaine sur treize ne trouvent **aucune ligne**. Zéro séjour orphelin, zéro code hors nomenclature, zéro acte hors de la fenêtre de son séjour. La tentation était de ne pas écrire ces règles, ou de les écrire sans les éprouver.

Elles sont donc exercées par injection : `tests.demontrer qualite` fabrique les lignes fautives que la source ne contient pas, vérifie ce que silver en fait, puis remet l'entrepôt en état. C'est ce qui a permis de découvrir que deux de ces règles étaient fausses — celle du libellé ci-dessus, et une autre plus bas.

Ce que nous en retenons. « Ce contrôle ne remonte rien » et « ce contrôle est cassé » se ressemblent parfaitement sur un tableau de bord. Seule une injection les distingue.

Certains défauts ne se voient qu'en ouvrant le produit fini

Trois défauts de ce rapport n'ont été trouvés qu'en lisant le PDF produit : l'encadré qui justifie l'absence de la Neurologie était **tronqué en plein milieu de sa phrase**, les douze captures ne s'affichaient pas du tout — les chemins étaient relatifs au document, pas au HTML intermédiaire — et un en-tête de tableau se coupait en « Format ».

Les trois documents étaient valides. Le Markdown était juste, le HTML aussi, aucun test ne les regardait — parce qu'aucun test ne regardait ce qu'un lecteur voit.

Ce que nous en retenons. Relire le produit fini fait partie du travail. Un document, un tableau de bord et une capture d'écran sont des livrables : ils se vérifient à l'œil, faute d'un contrôle qui sache le faire.

Une montée de version n'est pas réversible

ClickHouse 25.8 vers 26.8 a paru sans risque : mêmes tables, même SQL, zéro vulnérabilité de part et d'autre. Le moteur a démarré, le pipeline a tourné, les 428 contrôles sont passés. Puis `tests.demontrer` a signalé que le cloisonnement ne se démontrait plus au niveau de l'interface : la 26.x renvoie les refus de droits en **HTTP 403**, statut sur lequel le pilote embarqué dans Metabase ne lit plus le message d'erreur.

Revenir en arrière a détruit l'entrepôt. Les parts écrites par 26.8 sont illisibles par 25.8 — tables métier vidées ou non attachées — et il a fallu supprimer le volume et tout reconstruire depuis le lake. La reconstruction a rendu les chiffres à l'identique, mais l'historique des exécutions est perdu.

Ce que nous en retenons. Un essai de montée de version se fait sur un volume jetable, jamais sur celui du projet. Et une compatibilité se vérifie sur ce que l'utilisateur voit, pas seulement sur ce que les tests couvrent : ici le moteur avait raison de refuser, c'est l'affichage du refus qui était perdu.

Un client ne doit pas supposer le format de ce qu'il reçoit

Le client Metabase appelait `json.loads` sur toute réponse non vide. Or `/api/setting/<clé>` renvoie la valeur brute — `fr`, sans guillemets. La `JSONDecodeError` remontait nue jusqu'au gestionnaire d'échec inattendu : code de sortie 2 pour une réponse parfaitement valide.

Le défaut dormait depuis l'écriture du module. Aucun appel n'empruntait ce chemin. Il est apparu à la **deuxième** exécution après l'ajout d'un réglage — la première trouvait le réglage absent, donc `null`, donc du JSON.

Ce que nous en retenons. Une frontière avec un système extérieur doit tolérer ce qu'elle n'attend pas. Et un défaut qui n'apparaît qu'au second passage est la signature d'un état supposé plutôt que lu.

Une remise en état ne remet en état que ce qu'elle sait recharger

Les actes de démonstration étaient injectés au 28 août, comme ceux du monitoring. Or la remise en état ne recharge que les partitions **dont la source existe**, et le flux d'actes n'a de dépôt qu'au 29 : ces lignes survivaient à la démonstration, indestructibles par `eds.run --tout`, et invisibles au contrôle « aucune ligne de démonstration ne subsiste » qui ne les cherchait pas encore.

Le contrôle de propreté et l'injection avaient été écrits séparément, chacun juste de son côté.

Ce que nous en retenons. Une propriété d'idempotence porte sur un périmètre, et ce périmètre doit être énoncé. Ici : « les partitions dont la source existe », pas « l'entrepôt ».

Un avertissement qui décrit le normal cesse d'être lu

Le pipeline émettait 57 avertissements `source absente` par exécution complète — 28 pour les actes, déposés une seule fois, 26 pour les patients, qui sont un snapshot. Aucun ne signalait une anomalie : tous décrivaient un calendrier de dépôt irrégulier **par conception**.

Le motif préexistait, sous le seuil où l'on s'en aperçoit. L'évolution l'a doublé, et l'a rendu visible. Le journal du projet donne la mesure exacte de ce que cela coûte : **7 421 des 7 422 avertissements jamais émis** portaient ce seul motif. Le seul avertissement réel de toute l'histoire du dépôt — une connexion Metabase en double — était noyé dedans.

Ce que nous en retenons. Un `WARNING` qui décrit une situation normale use le signal : l'exploitant qui en voit 57 prend l'habitude de ne pas les lire, et rate le jour où il y en a un vrai. Le motif est passé en `INFO` et `WARNING` est redevenu le niveau de ce qui mérite d'être lu (§ 11). La leçon n'est pas le réglage lui-même, c'est qu'un niveau de journalisation se décide sur ce que le message signifie pour celui qui le lira, jamais sur la gravité apparente du mot.

Ne rien casser ne coûte rien — si l'on s'y est préparé avant

Le sujet d'évolution demandait de « faire évoluer sans tout refaire, sans rien casser ». Les deux moitiés n'ont pas coûté la même chose.

Ne rien casser n'a rien coûté, et c'est le résultat le plus net. Aucun indicateur de la première livraison n'a bougé — la DMS en réanimation vaut toujours 9,05 jours, le case-mix et la prévalence sont inchangés, `tests.verifier conformite` reste au vert face aux valeurs de référence de l'intervenant. Non parce que la modification était petite, mais parce que l'entrepôt était déjà instrumenté : les contrôles existaient **avant** d'être nécessaires.

Ne pas tout refaire, en revanche, s'est décidé. La tentation était réelle de traiter l'évolution comme un projet séparé — un second entrepôt, un second code. C'est précisément ce qui aurait rendu la non-régression indémontrable : deux codes différents donnant deux résultats ne prouvent rien.

Ce que nous en retenons. La non-régression ne se vérifie pas après coup, elle se prépare. Ce qui l'a rendue démontrable ici, ce sont des contrôles écrits des semaines avant qu'on en ait besoin — et un seul entrepôt, augmenté plutôt que dupliqué.